



RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Unit 3

Analog and Digital IOs

Unit 3: Syllabus

Analog and Digital IOs:

ARM Cortex M4 MCUs, Memory organization, Reset & Clock Control, GPIO, Programming: interfacing LEDs and Push buttons, Analog to digital converters (ADC), Successive Approximation ADC, Programming and interfacing an analog sensor, Digital to Analog Converter (DAC), Programming

ARM Cortex- M4 MCUs

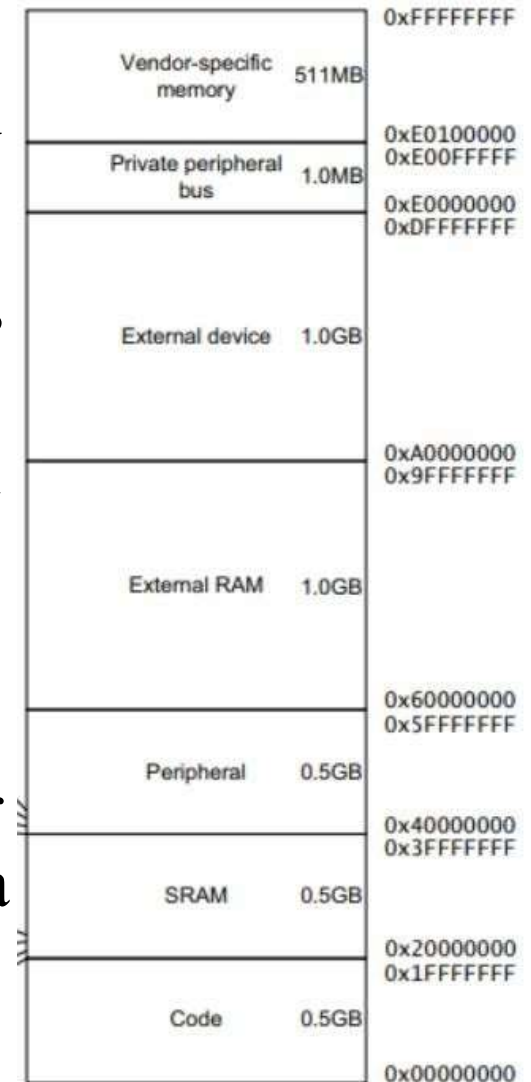
Leading Manufactures

- **STMicroelectronics (ST):**
 - STM32F4 series
- **NXP Semiconductors:**
 - LPC4300 series, Kinetis K series
- **Texas Instruments (TI):**
 - Tiva C Series
- **Renesas:**
 - RA4, RA6 Series.
- **Nuvoton Technology:**
 - M463,M467,M471....series
- **Infineon Technology Pvt. Ltd:**
 - XMC4000 series
- **Silicon Labs:**
 - EFM32 Giant Gecko series 1
- **Toshiba:**
 - TX04 series
- **Microchip Technology:**
 - SAM4 series
- **Analog Devices:**
 - MAX series

Memory Organization

- The Cortex-M processors have 32-bit memory addressing and therefore have 4GB memory space.
- The memory space is unified, which means instructions and data share the same address space.
- Multiple bus interfaces to allow concurrent instructions and data accesses
- The memory map of cortex M4 CPUs is shown in figure.

Note: A memory map is a diagram or representation that illustrates the organization of a computer system's memory.



Memory Organization...

- As shown in the figure, the flash program memory on the microcontroller chip is mapped code region.
- The data memory is mapped to locations shown as SRAM in figure.
- The registers of different peripherals available in the microcontroller are mapped to peripheral region.
- External memory interface like SD card/SRAM/NAND Flash is mapped region external SRAM and external device.
- Private peripheral bus indicate the region of locations used for NVIC registers, FPU registers, SysTick Timer registers, Trace and debug registers, etc.
- Vendor specific memory implementation varies with different vendors.

Memory Map of STM32F407VG

Microcontroller support:

- Up to 1 MB of Flash memory(Program memory)
- Up to 192 KB of SRAM(Data memory) including 64 KB of CCM (core coupled memory) data RAM
- 512 bytes of OTP(one time programmable) memory
- Flexible Static Memory Controller(FSMC) supporting Compact Flash(similar to SD card), SRAM, PSRAM, NOR and NAND memories.

Note: PSRAM(Pseudo Static RAM) is
DRAM(Dynamic RAM) combined
with a self-refresh circuit.
NAND and NOR are type of FLASH memories.

0xFFFF FFFF	512-Mbyte block 7 Cortex-M4's internal peripherals
0xE000 0000 0xDFFF FFFF	512-Mbyte block 6 Not used
0xC000 0000 0xBFFF FFFF	512-Mbyte block 5 FSMC registers
0xA000 0000 0x9FFF FFFF	512-Mbyte block 4 FSMC bank 3 & bank4
0x8000 0000 0x7FFF FFFF	512-Mbyte block 3 FSMC bank1 & bank2
0x6000 0000 0x5FFF FFFF	512-Mbyte block 2 Peripherals
0x4000 0000 0x3FFF FFFF	512-Mbyte block 1 SRAM
0x2000 0000 0x1FFF FFFF	512-Mbyte block 0 Code
0x0000 0000	

Core Coupled Memory

- A special memory that provide high performance and Zero wait state access (faster access) to memory and hence accessing is faster as compared to execution from flash program memory.
- It is was included in the microcontrollers for use in scenarios that involve real-time and computation-intensive routines like real-time DSP (digital signal processing).

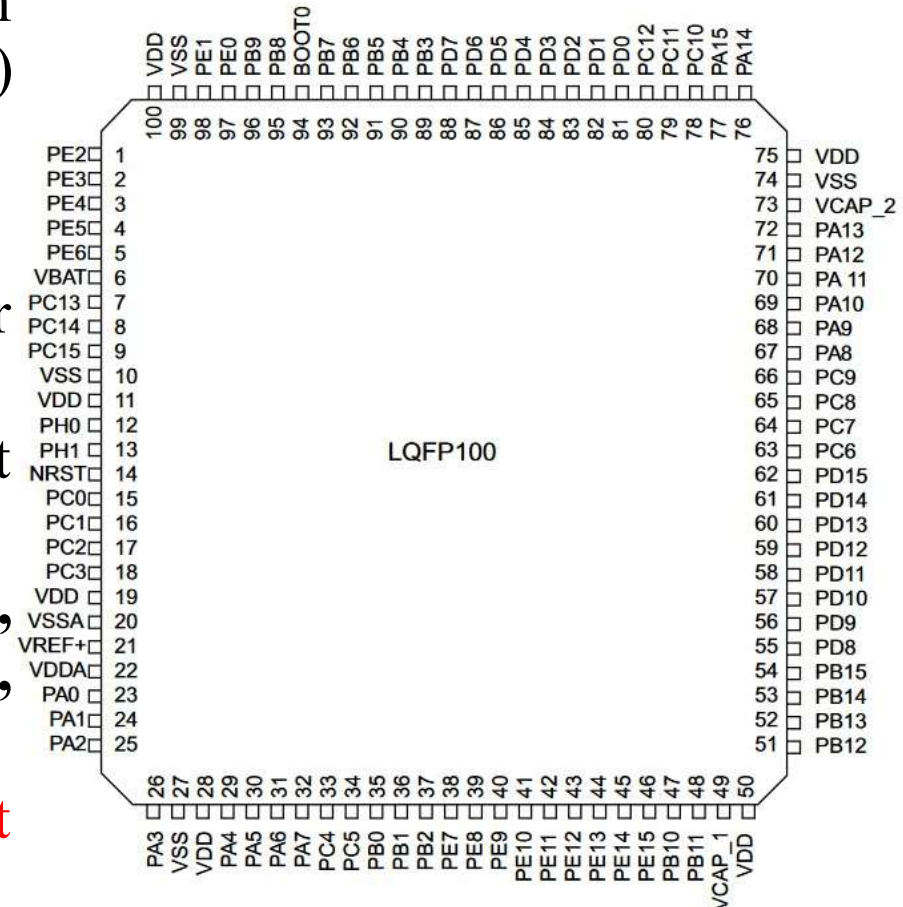
Flexible Static Memory Controller

- This provides external memory support through controllers. Using registers of FSMC, configuration of external memory chips can be carried out.

Reset and Clock Controller(RCC)

- Discovery board used in lab is hosting 100 pin Low Profile Quad Flat Package(LQFP) STM32F407VGT6 IC.

100 LQFP pin diagram



Reset

- When MCU is reset, all registers are set to their reset state and **Reset_Handler** get executed.
- The RESET service routine vector is fixed at address **0x0000 0004** in the memory map.
- The Reset sources are low signal on **NRST pin**, **Independent Watch Dog Reset(IDWG)**, **Power on Reset**, **Brown out Reset**

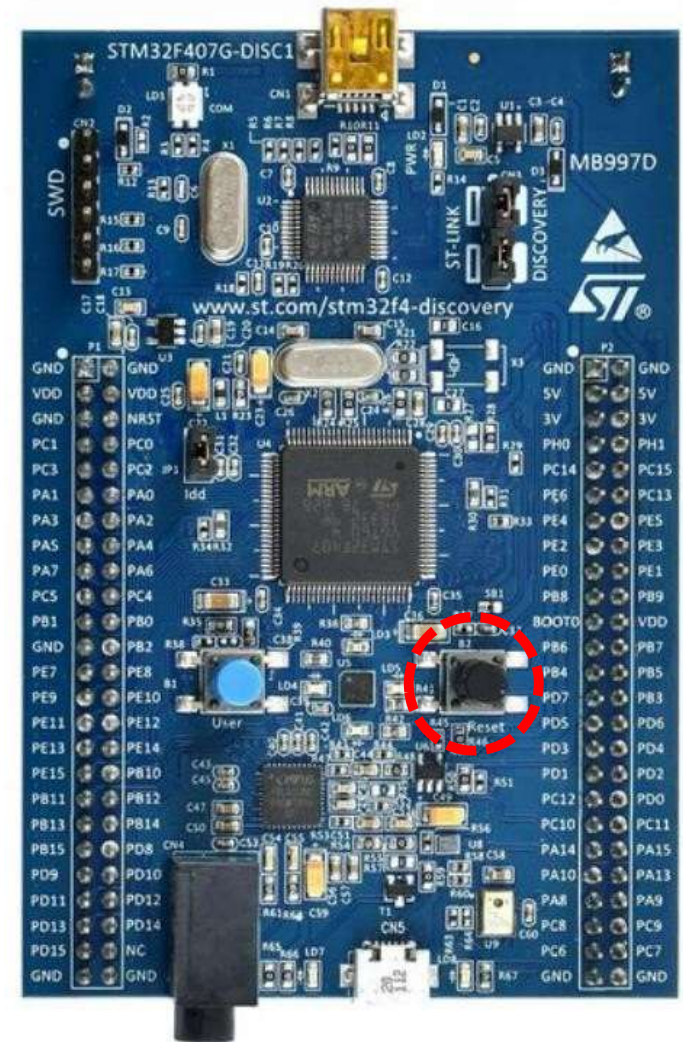
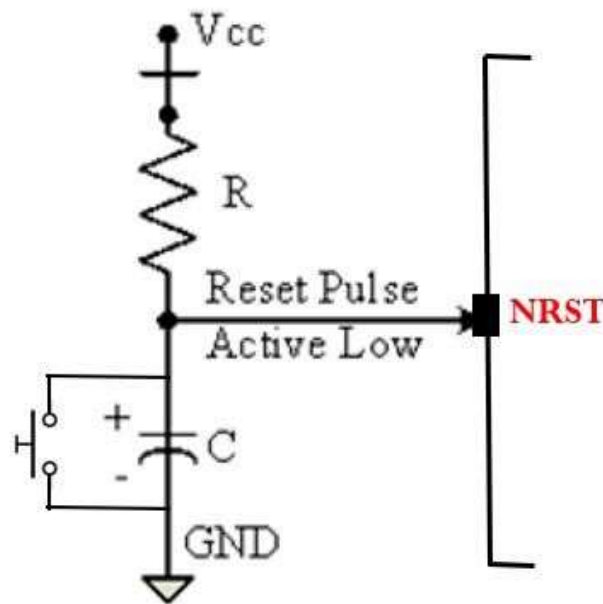
Note: Other resets STM32F407VG are not considered.

Activity

Exploration of Reset Handler in assembly program and C program

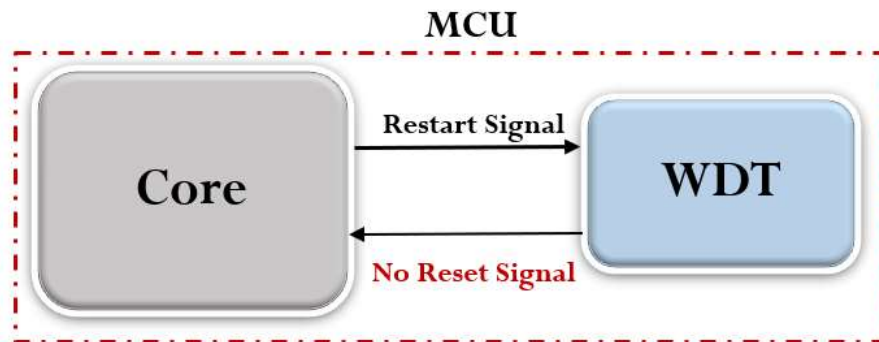
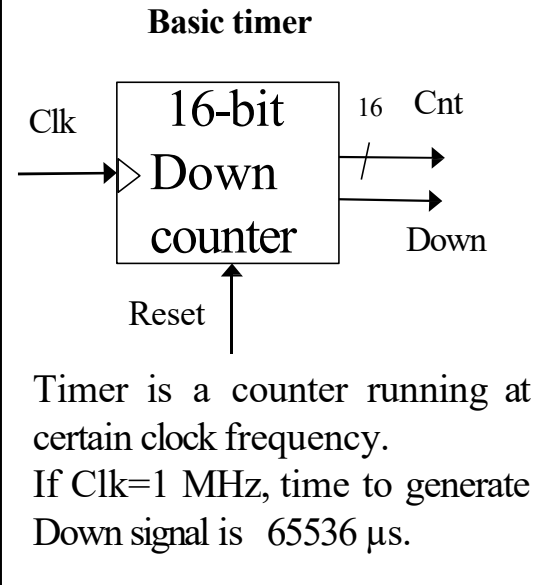
- **NRST** pin must be held low to initiate the reset operation.
- **Power on Reset/Manual Reset:** The reset can be initiated when power supply is applied/ A push button is provided to initiate manual reset operation.
- Schematics is shown in figure below.

RCC....

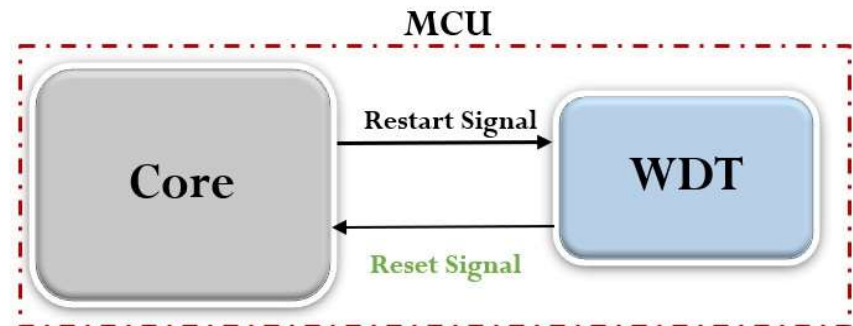


Independent Watch Dog Reset(IDWG)

- A watchdog reset typically refers to the action taken by a watchdog timer when it detects that a system or device has failed or become unresponsive.
- If errors are transient(non recurring), a reset operation can be used to recover.
- The CPU has to send reset signal to watchdog timer periodically, otherwise watchdog timer generate reset signal.
- Operation of WDT reset is shown in figure below:



If core sends a restart signal at regular intervals, no reset signal from WDT

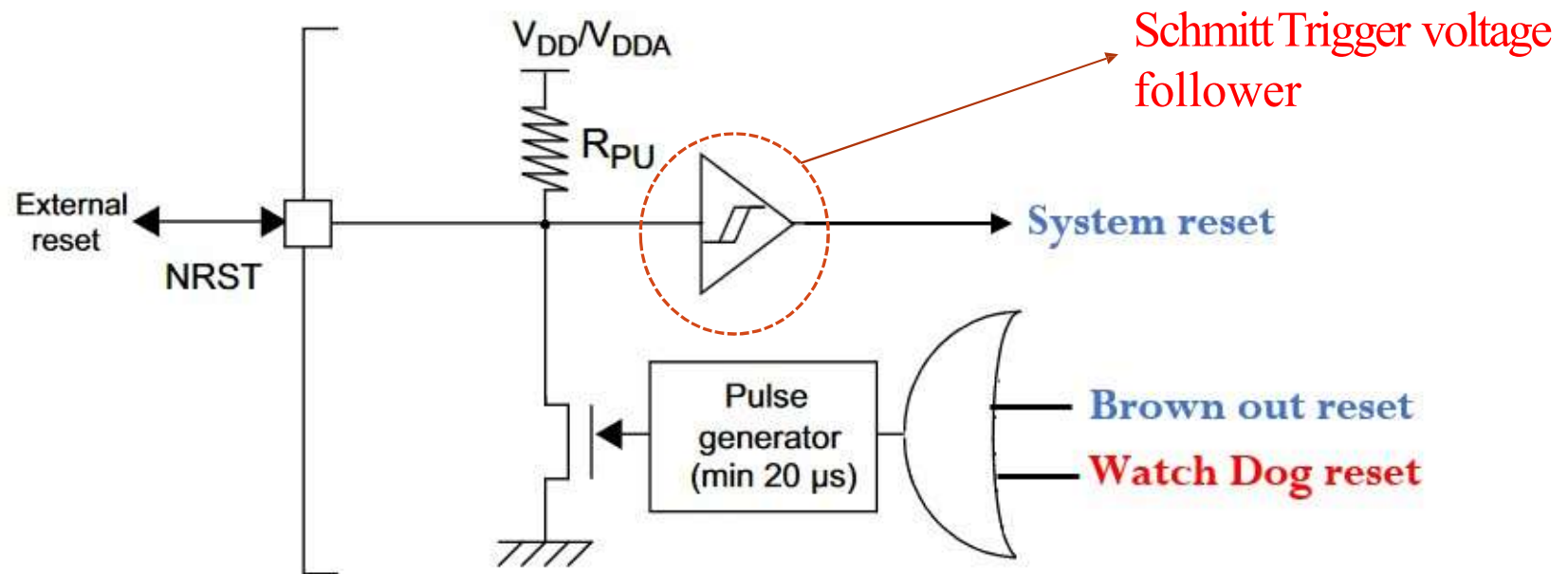


If core sends no restart signal at regular intervals, a reset signal from WDT

Brown out Reset

- This is a protection circuit inside MCU which detects when supply voltage(VDD) goes below certain level and puts the device into a reset state to ensure proper startup when power returns.
- The MCU behavior is unpredictable(not reliable) when supply voltage falls below certain threshold.
- Brownout detection circuit holds the MCU in reset as long as supply voltage is below certain threshold.

Simplified block diagram of Reset Circuits

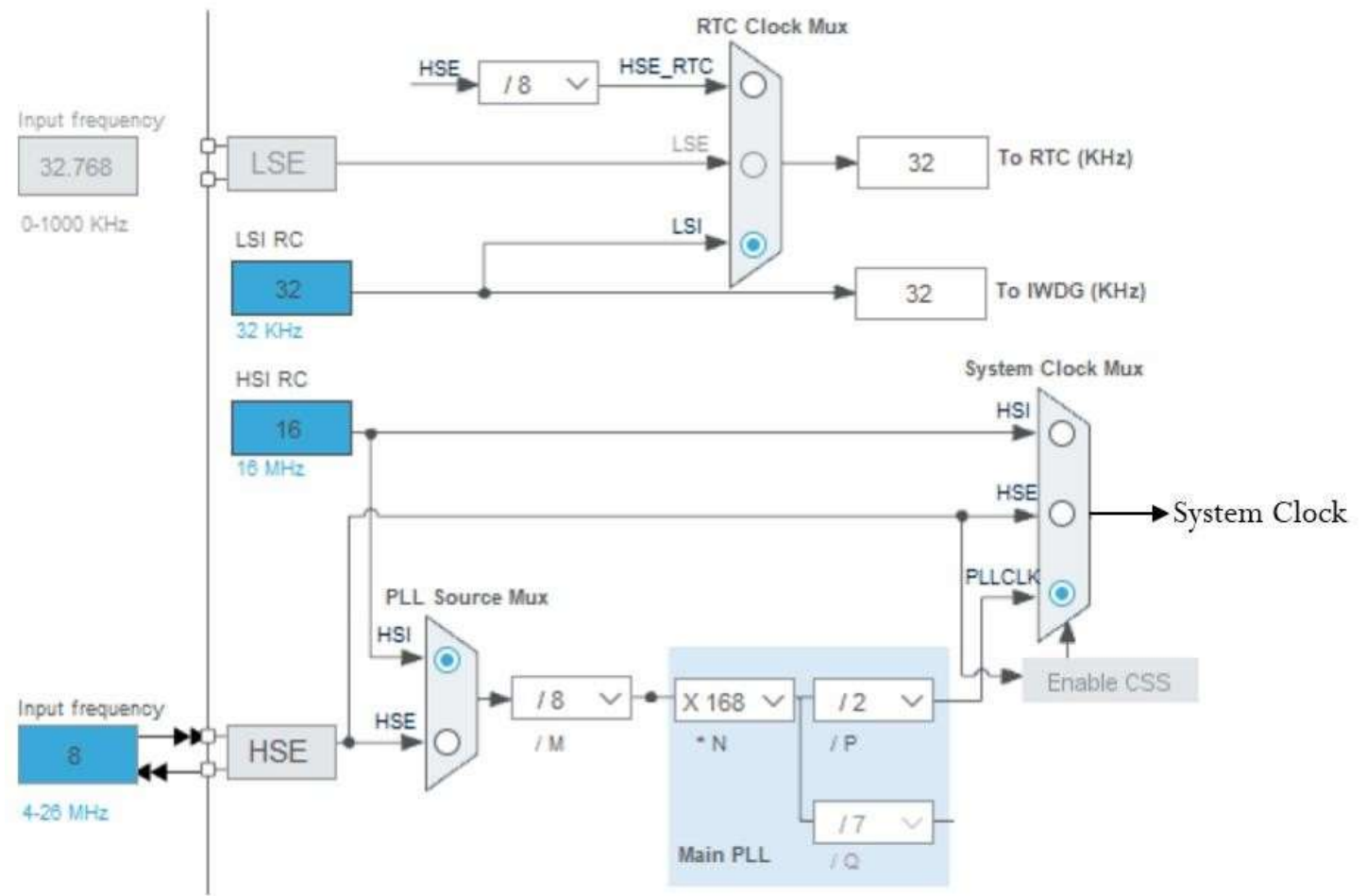


Clocks

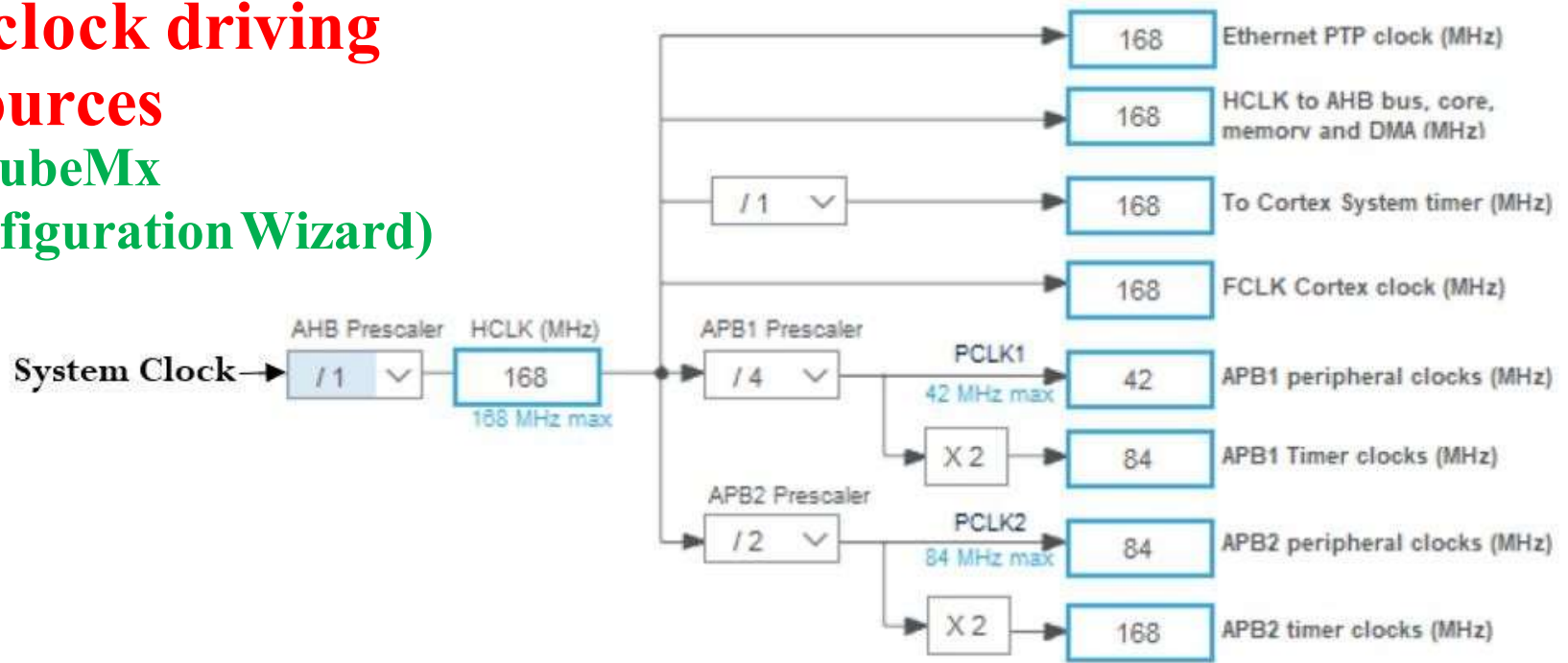
- Three different clock sources can be used to drive the system clock (SYSCLK):
 - HSI(High Speed Internal) oscillator clock: 16 MHz Internal RC Oscillator
 - HSE(High Speed External) oscillator clock:4-26 MHz External LC oscillator
 - Main PLL (Phase Locked Loop) clock:This system act as frequency multiplier or divider to generate different clock sources.
- Two secondary clock sources are provided:
 - 32 kHz low-speed internal RC (LSI RC) which drives the independent watchdog and RTC clock.
 - 32.768 kHz low-speed external crystal (LSE crystal) which optionally drives the RTC clock.

RCC...

**Clock Sources
on board used
in lab**
(STM32CubeMx
Clock configuration
Wizard)



**System clock driving
other sources
(STM32CubeMx
Clock configuration Wizard)**



Note: Clock sources to different units

GPIOs: HCLK, Core: FCLK, DAC:APB1 Peripheral clock,
ADC:APB2 peripheral clock

Important point to remember !

All configurations in STM32CubeMx
writes configuration data into
registers.



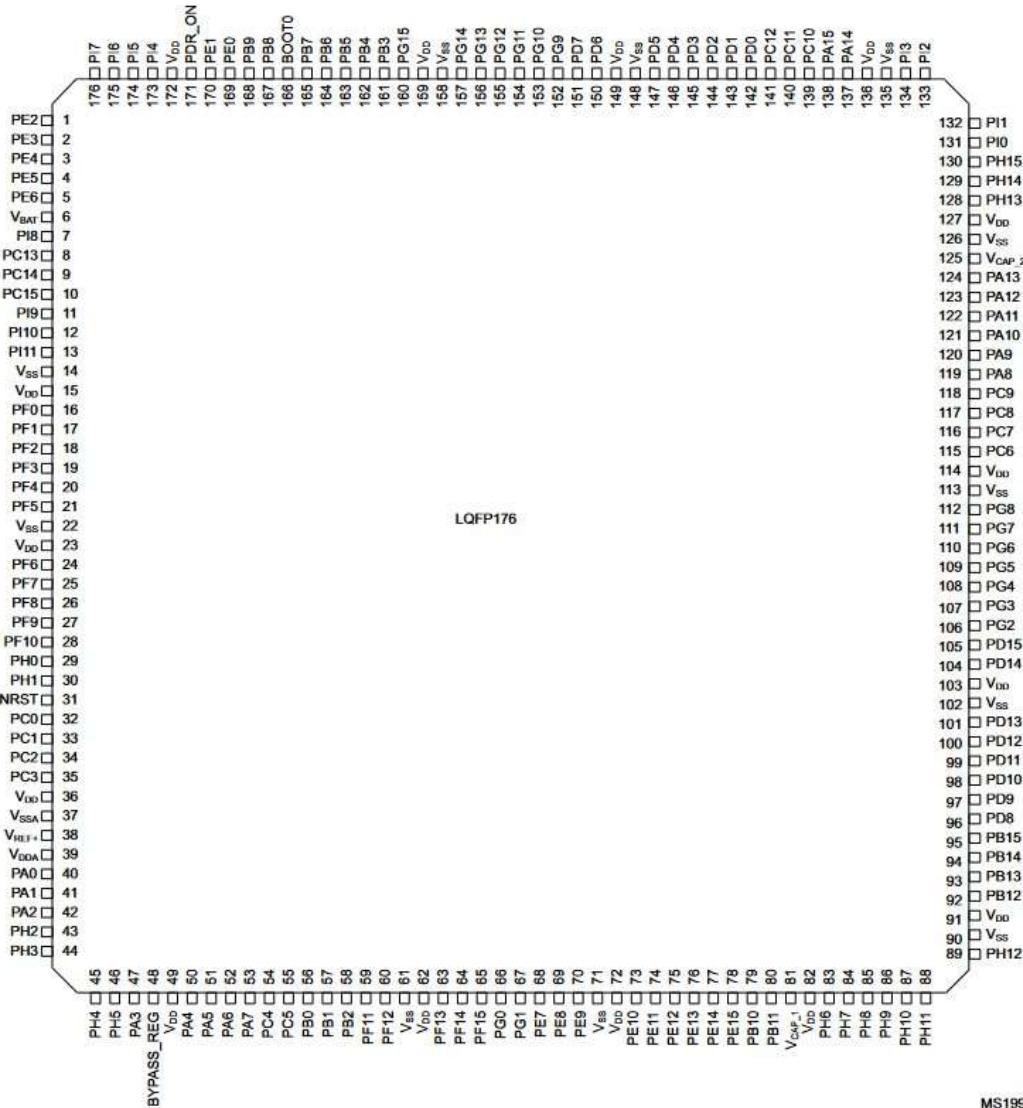
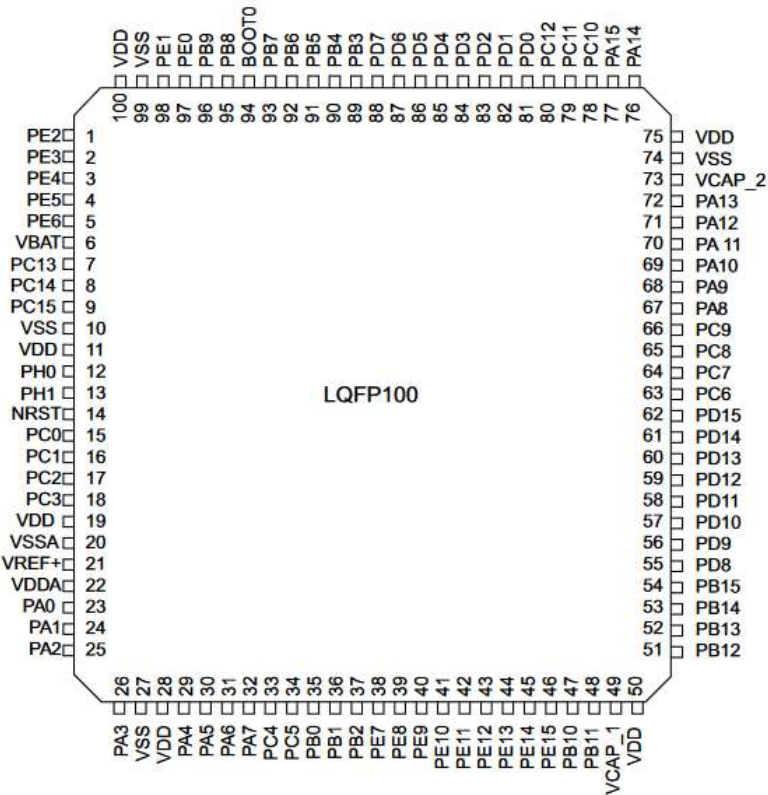
General Purpose IOs(GPIOs)

- Most of the pins of the STM32 can be configured as input or output and may be connected to either the GPIO ports or “alternative functions”.
- As a standard naming convention, the pins are called by their GPIO function.
Example: PA0 (bit 0 of port A) or PB9 (bit 9 of port B).
- Maximum 16 pins in each port. Number of ports varies depending in packaging type.

Example: 176 pin LQFP: **Port A to Port I**

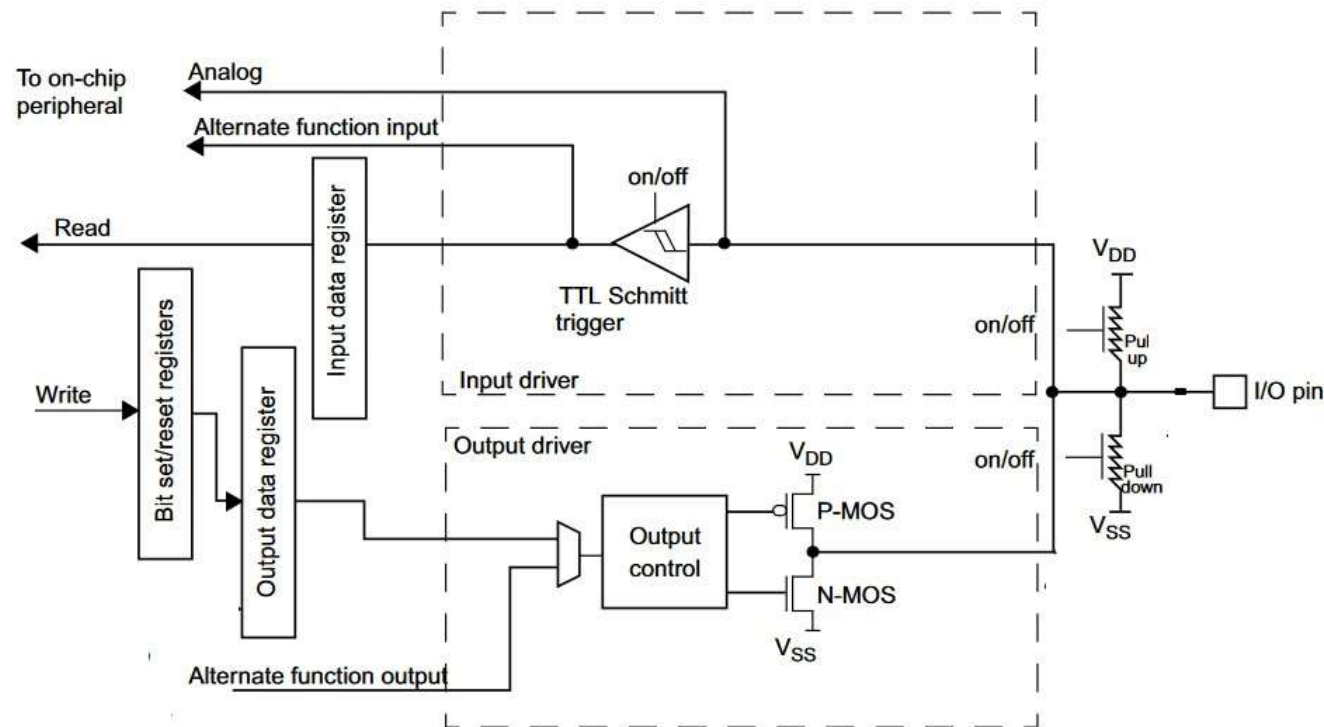
100 pin LQFP: **Port A to Port E**

GPIOs...



Basic Structure IOs

- The figure shows basic structure of IO pin.
- The each pin has input driver and output driver.
- These drivers used to configure the pin as per requirement.
- The alternate input and output functions varies with port to port.



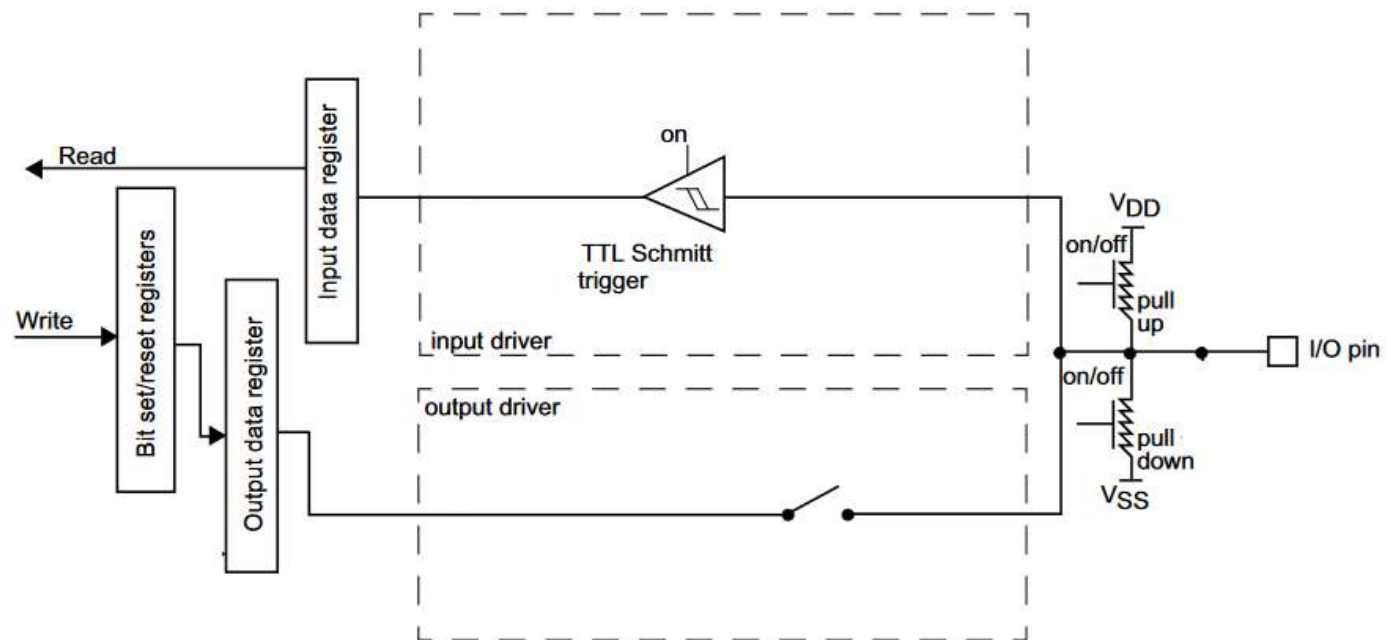
Observe the usage of registers BSRR, ODR and IDR

Input Operation

GPIOs...

When the I/O port is programmed as Input:

- The output buffer is disabled and Schmitt trigger input is activated.
- The pull-up and pull-down resistors are activated or not, depending on the value in the GPIOx_PUPDR(GPIO port pull-up/pull-down) register.
- The data present on the I/O pin are sampled into the input data register every AHB1 clock cycle.
- The pin can be **floating** if pull up and pull down are disabled.



STM32CubeMx Configurations for input port:

PA0-WKUP Configuration :

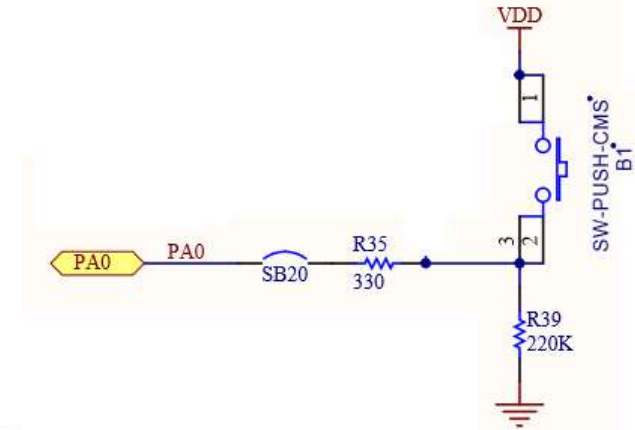
GPIO mode	Input mode
GPIO Pull-up/Pull-down	No pull-up and no pull-down
User Label	

Autogenerated Configurations for input port:

```
GPIO_InitTypeDef GPIO_InitStructure = {0};  
  
/*Configure GPIO pin : PA0 */  
GPIO_InitStructure.Pin = GPIO_PIN_0;  
GPIO_InitStructure.Mode = GPIO_MODE_INPUT;  
GPIO_InitStructure.Pull = GPIO_NOPULL;  
HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);
```

Input Operation...

What is the configuration for Push Button connection?



```
typedef struct  
{  
    uint32_t Pin;  
    uint32_t Mode;  
    uint32_t Pull;  
    uint32_t Speed;  
    uint32_t Alternate;  
}GPIO_InitTypeDef;
```

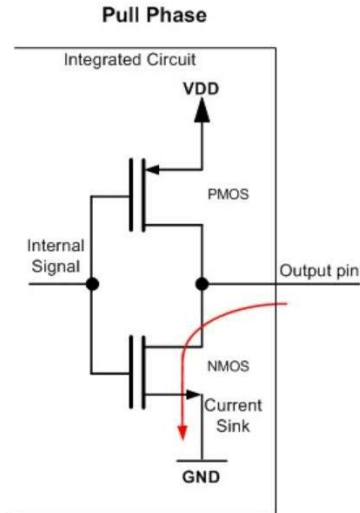
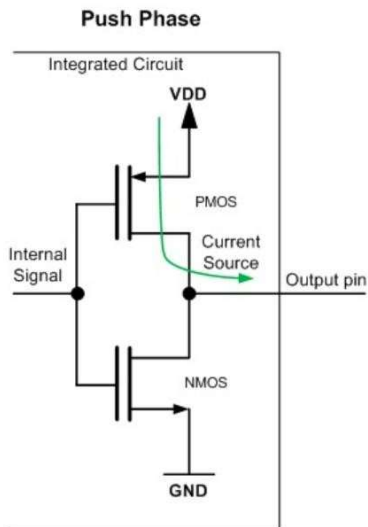
Output Operation

GPIOs...

When the I/O port is programmed as output:

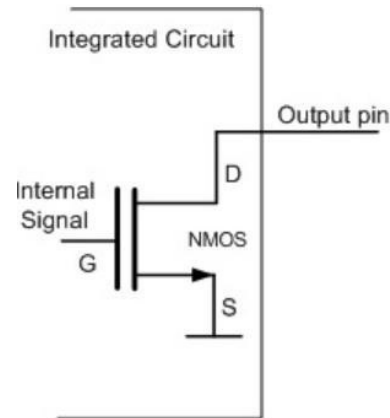
- The output buffer is enabled.
- The port pin can be configured Push pull or Open drain with pull ups enabled or disabled.

Push Pull:



Pin state=0

Open drain:



Internal or external pull up is required when pin state is Hi-Z

When,
NMOS is on, pin state=0
NMOS is off, pin is floating(Hi-Z state)
*Hi-Z is high impedance state

STM32CubeMx Configurations for output port pin PD15:

Output Operation...

PD15 Configuration :

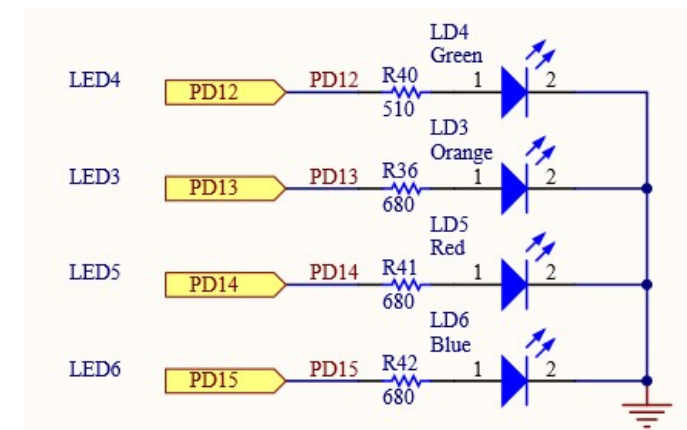
GPIO output level: Low

GPIO mode: Output Push Pull

GPIO Pull-up/Pull-down: No pull-up and no pull-down

Maximum output speed: Low

What is the configuration for LED connection?



Can we select output open drain to connect LED?

What is the advantage of external pull up?

Output Operation...

Autogenerated Configurations for output port:

```
/*Configure GPIO pins : PD12 PD13 PD14 PD15 */
GPIO_InitStruct.Pin = GPIO_PIN_12|GPIO_PIN_13|GPIO_PIN_14|GPIO_PIN_15;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
```

● Speed Configurations for output port(Varies from device to device):

```
#define GPIO_SPEED_FREQ_LOW          0x00000000U  /*!< IO works at 2 MHz
#define GPIO_SPEED_FREQ_MEDIUM      0x00000001U  /*!< range 12,5 MHz to 50 MHz
#define GPIO_SPEED_FREQ_HIGH        0x00000002U  /*!< range 25 MHz to 100 MHz
#define GPIO_SPEED_FREQ_VERY_HIGH  0x00000003U  /*!< range 50 MHz to 200 MHz
/**
```

Registers for Operation

● GPIO port mode register (GPIOx_MODER)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 2y:2y+1 **MODERy[1:0]**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O direction mode.

00: Input (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

Registers for Operation...

- GPIO port input data register (GPIOx_IDR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **IDRy**: Port input data (y = 0..15)

These bits are read-only and can be accessed in word mode only. They contain the input value of the corresponding I/O port.

Registers for Operation...

GPIO port bit set/reset register (GPIOx_BSRR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:16 **BRy**: Port x reset bit y (y = 0..15)

These bits are write-only and can be accessed in word, half-word or byte mode. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Resets the corresponding ODRx bit

Note: If both BSx and BRx are set, BSx has priority.

Bits 15:0 **BSy**: Port x set bit y (y= 0..15)

These bits are write-only and can be accessed in word, half-word or byte mode. A read to these bits returns the value 0x0000.

0: No action on the corresponding ODRx bit

1: Sets the corresponding ODRx bit

RCC Registers to enable clock for GPIOs

RCC AHB1 peripheral clock enable register (RCC_AHB1ENR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved	OTGH S ULPIE N	OTGH SEN	ETHM ACPTP EN	ETHM ACRXE N	ETHM ACTXE N	ETHMA CEN	Reserved		DMA2E N	DMA1E N	CCMDAT ARAMEN	Res.	BKPSR AMEN	Reserved	
	rw	rw	rw	rw	rw	rw			rw	rw			rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved			CRCE N	Reserved			GPIOE N	GPIOH EN	GPIOG EN	GPIOF E N	GPIOEEN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
			rw				rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 0 **GPIOAEN**: IO port A clock enable

Set and cleared by software.

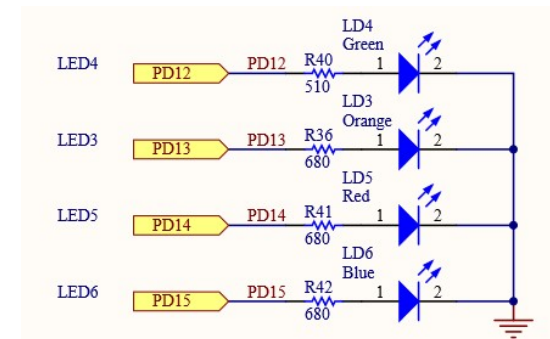
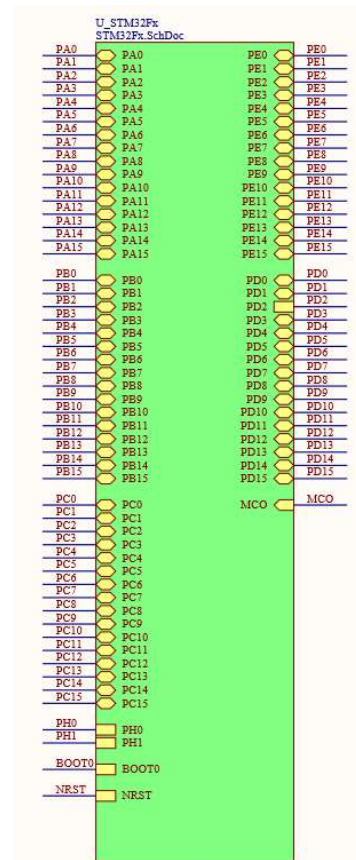
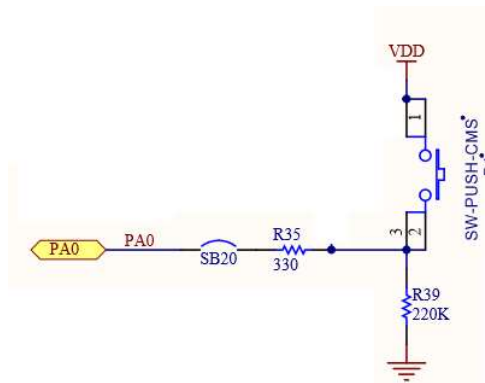
0: IO port A clock disabled

1: IO port A clock enabled

Port Programming

- Design STM32F407VG based to connect push button and LED. Write program to read Push button status and toggle LED.

Interfacing diagram:



Program(register programming)

```
#include "stm32f4xx.h"          /*Header file with register declarations*/

void delayMs(int n);

void main() {
    RCC->AHB1ENR |= 9;           /* enable Clock to GPIOA & GPIOD */
    GPIOD->MODER |= 0x40000000; /*Set PD15 as GPIO output & PA0 act as input by
                                default*/

    While(1){
        if(GPIOA->IDR & 0x0000 0001==1)
            GPIOD->BSRR=1<<15;   /*set bit 15 to make PD15 to 1*/
        else
            GPIOD->BSRR=1<<31;    /*set bit 31 to make PD15 to 1*/
    }
}
```

Program(using STM32CubeMx):

Port Programming....

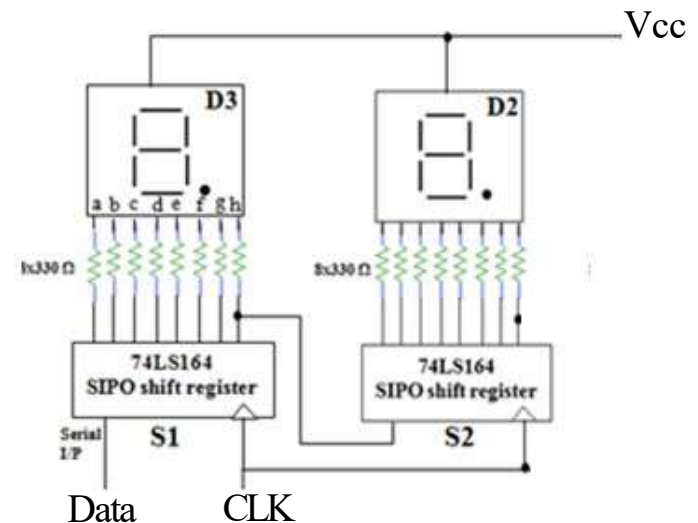
- Assume that, STM32CubeMx is used and PA0 is configured as GPIO_Input(No Pullup/Pull down) and PD15 as GPIO_Output(Output push pull and No Pullup/Pull down) and HAL is generated.

Application code:

```
#include "main.h"
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    while (1)
    {
        if(HAL_GPIO_ReadPin(GPIOA,GPIO_PIN_0)==GPIO_PIN_SET)
            HAL_GPIO_WritePin(GPIOD, GPIO_PIN_15,GPIO_PIN_SET);
        else
            HAL_GPIO_WritePin(GPIOD,GPIO_PIN_15,GPIO_PIN_SET);
    }
}
```

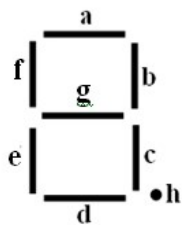
Question 1

The diagram below shows an interface that provide two digit seven segment display driven by the outputs of two cascaded serial-in-parallel-out shift registers. Data to be displayed is transmitted serially (first segment H). Each bit is clocked into the shift registers by providing common clock. Data in first shift register appears at second after eight clock pulses. Provide a push button to start the counting operation.



Question 1...

- The seven segment type is common anode type. The display codes are obtained as follows.



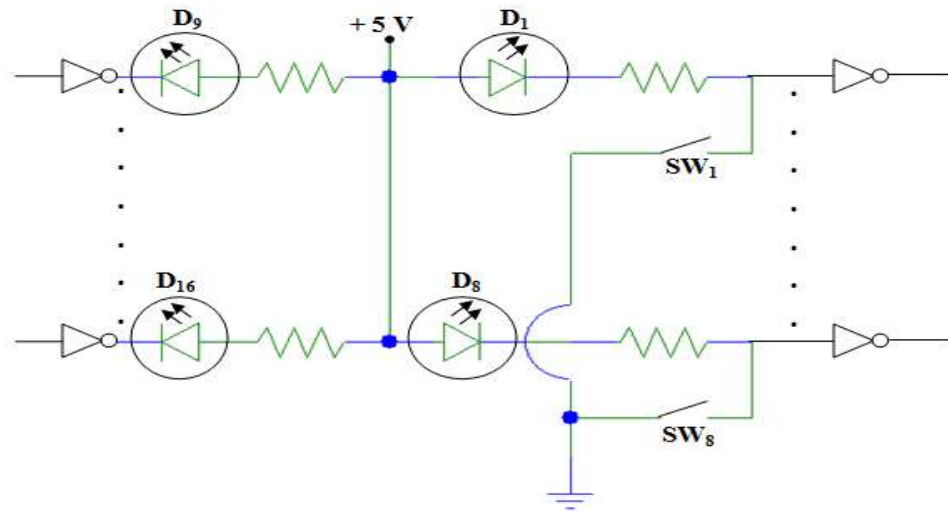
Segment values to display 3

Segment	A	B	C	D	E	F	G	H
Bit	7	6	5	4	3	2	1	0
Input	0	0	0	0	1	1	0	1
Hex Code	0				D			

- Seven segment display codes: 03H,9FH,25H,0DH,99H,49H,41H,1FH,01H,09H,
Design STM32F407VG system interface seven segment display & realize two digit BCD counter.

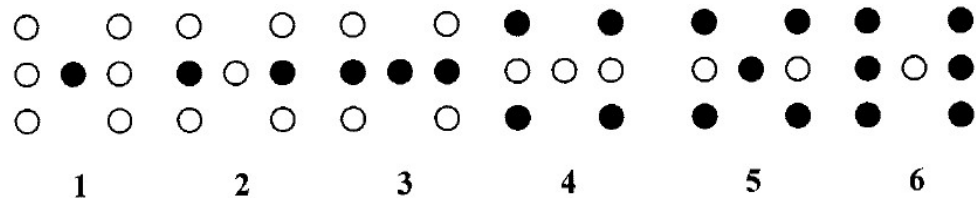
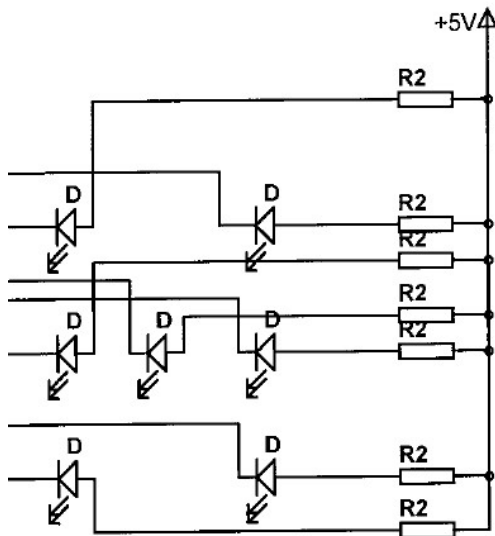
Question 2

- The diagram below shows an interface card with eight DIP switches with status indicating LEDs D_1 & D_8 . The card also features 8 LEDs (D_9 - D_{16}) controlled by external devices. Design a suitable scheme to interface card to STM32F407VG. Write a program to read the status of switches & display the same on LEDs D_9 - D_{16} .



Question 3

- In the diagram shown below, the seven LEDs are mounted to emulate the dots on real dice. The pattern displayed for different numbers is shown. As in real dice, first row can have up to two LEDs on (Corresponding to two dots on a dice), second row have up to three LEDs on and third row have up to two LEDs on.



Question 3...

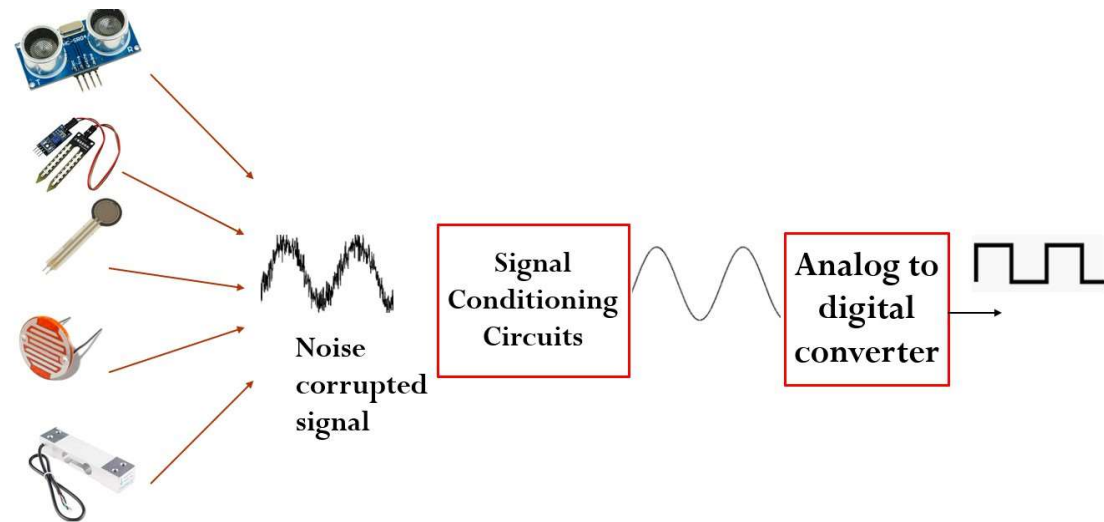
A random dice number is obtained by scanning push button switch as follows. If the switch is not pressed, the number is incremented between 1 and 6. When ever the push button is pressed, the current value of the number is read and this value is used as new dice number. The new number is displayed on LED for 2 seconds.

Design STM32F407VG system with above specifications and write C program to emulate dice operations.

Analog to Digital Converter (ADC)

- ADC is an electronic component or circuit that converts analog signals into digital representations.
- Analog signals are continuous voltage or current variations that represent real-world phenomena like sound, temperature, light, pressure, and more.
- Some time measured signals are noisy and hence to be corrected using signal conditioning circuits.

E.g: High pass filter to remove high frequency noise



ADC...

- An ADC has n-bit resolution where n is 8, 10, 12, 16 or even 24 bits. The higher resolution, ADC provides smaller step size, where step size is the smallest change in input analog voltage, an ADC can detect.
- If the full scale input voltage required to cause a digital output of all 1's is V_{IFS} , then resolution (step size) can be given as,

$$\text{Step size} = V_{IFS} / 2^n$$

- It is observed that, there is an unavoidable uncertainty about the exact value of input voltage for all digital output. This uncertainty is specified as quantization error. Its value is $\pm \frac{1}{2}$ LSB.

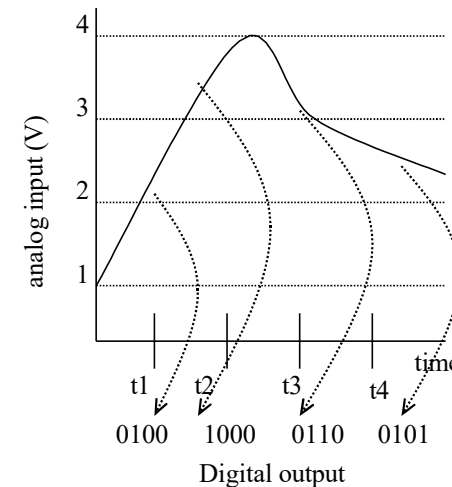
ADC...

- The output(d) of ADC for an analog input voltage 'e' V is given as:

$$d = e / \text{Resolution}$$
- The figure shows input and output proportionality 4 bit ADC.

$V_{\max} = 7.5V$	1111
7.0V	1110
6.5V	1101
6.0V	1100
5.5V	1011
5.0V	1010
4.5V	1001
4.0V	1000
3.5V	0111
3.0V	0110
2.5V	0101
2.0V	0100
1.5V	0011
1.0V	0010
0.5V	0001
0V	0000

Proportionality

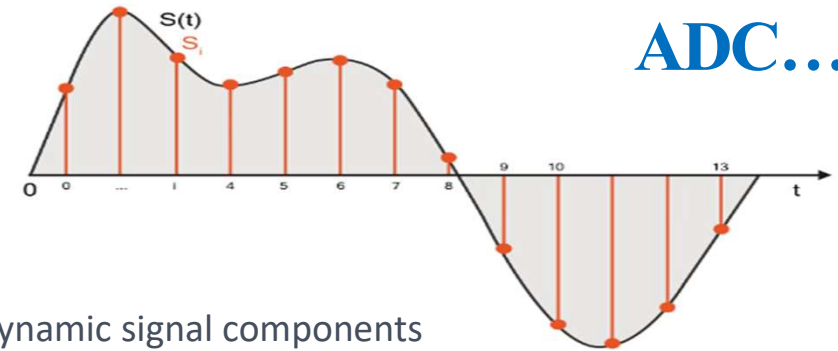


analog to digital

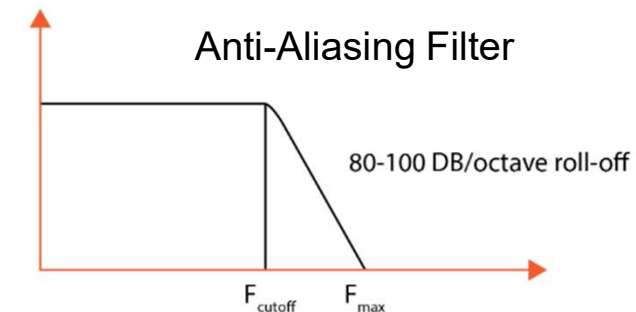
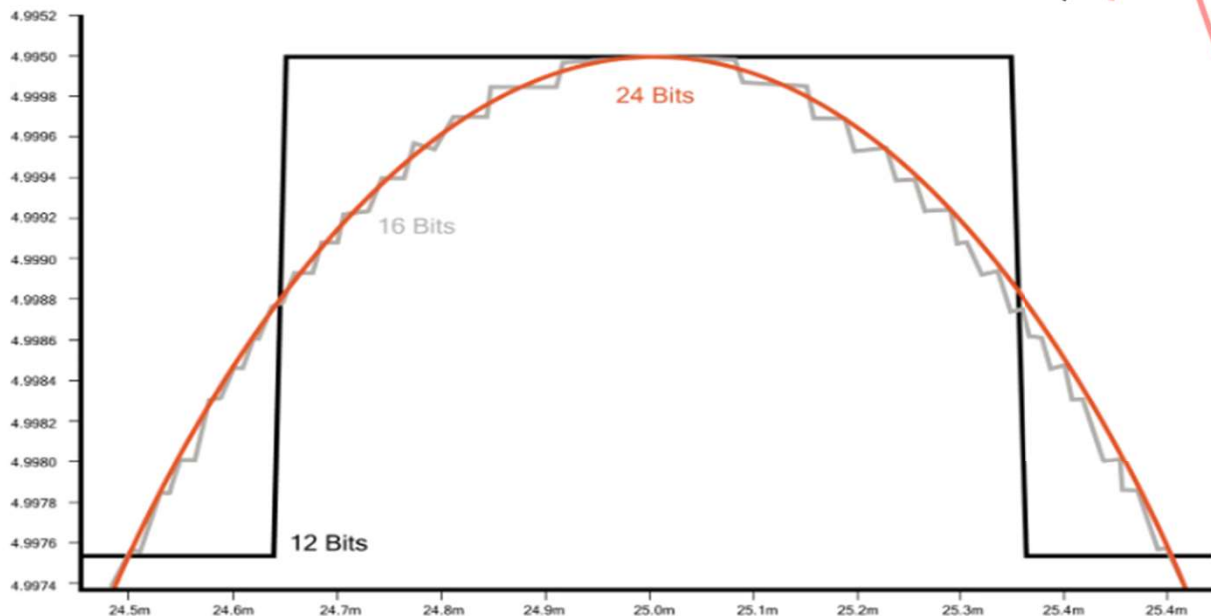
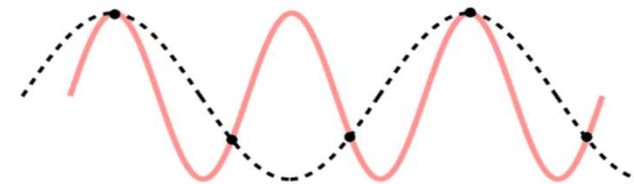
ADC...

There are really five major types of ADCs in use today:

- Successive Approximation (SAR) ADC
- Delta-sigma ($\Delta\Sigma$) ADC
- Dual Slope ADC
- Pipelined ADC
- Flash ADC

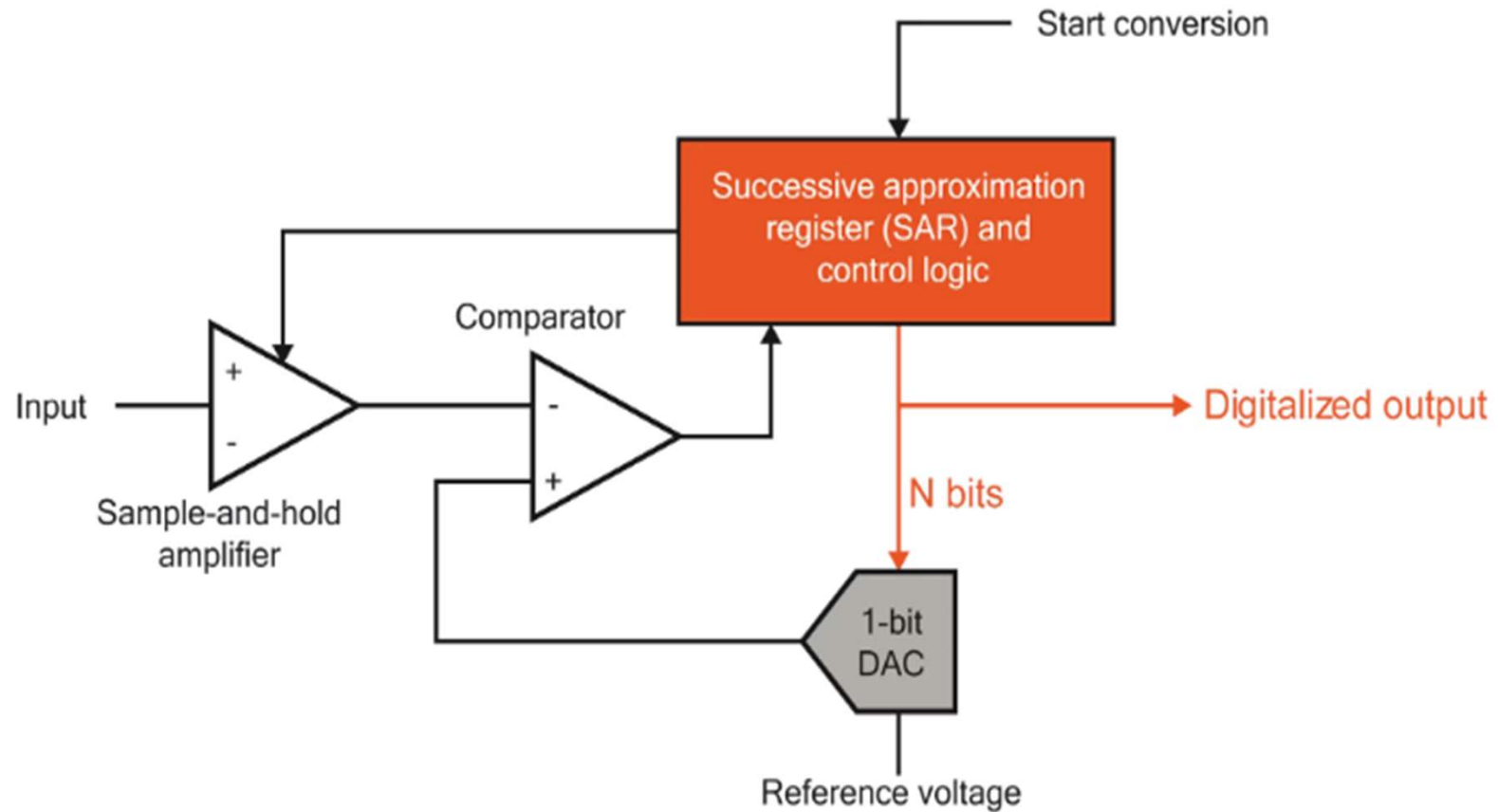


1. Missing vital dynamic signal components
2. Ending up with false ("alias") signals (if the system lacks anti-aliasing filter)



Successive Approximation ADC:

ADC...



Successive Approximation ADC:

ADC...

- Slower than flash type ADC but faster than other types. The input voltage is successively approximated to arrive at digital output.
- For the given an analog input signal whose voltage should range from 0($V_{\min}$) to 15($V_{\max}$) volts, and an 8-bit digital encoding is calculated for **5 volts** as follows.

Resolution: $15/256=0.058853$; Digital Encoding= $5/0.058853=85d=55h=01010101$

$$\frac{1}{2}(V_{\max} + V_{\min}) = 7.5 \text{ volts}$$
$$V_{\max} = 7.5 \text{ volts.}$$

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

$$\frac{1}{2}(5.63 + 4.69) = 5.16 \text{ volts}$$
$$V_{\max} = 5.16 \text{ volts.}$$

0	1	0	1	0	0	0	0
---	---	---	---	---	---	---	---

$$\frac{1}{2}(7.5 + 0) = 3.75 \text{ volts}$$
$$V_{\min} = 3.75 \text{ volts.}$$

0	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---

$$\frac{1}{2}(5.16 + 4.69) = 4.93 \text{ volts}$$
$$V_{\min} = 4.93 \text{ volts.}$$

0	1	0	1	0	1	0	0
---	---	---	---	---	---	---	---

$$\frac{1}{2}(7.5 + 3.75) = 5.63 \text{ volts}$$
$$V_{\max} = 5.63 \text{ volts}$$

0	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---

$$\frac{1}{2}(5.16 + 4.93) = 5.05 \text{ volts}$$
$$V_{\max} = 5.05 \text{ volts.}$$

0	1	0	1	0	1	0	0
---	---	---	---	---	---	---	---

$$\frac{1}{2}(5.63 + 3.75) = 4.69 \text{ volts}$$
$$V_{\min} = 4.69 \text{ volts.}$$

0	1	0	1	0	0	0	0
---	---	---	---	---	---	---	---

$$\frac{1}{2}(5.05 + 4.93) = 4.99 \text{ volts}$$

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Successive Approximation ADC:

ADC...

Pro's

- Simple circuit with only one comparator needed
- Higher sample rates are possible compared to delta-sigma ADCs
- Handles natural and unnatural waveshapes well

Cons

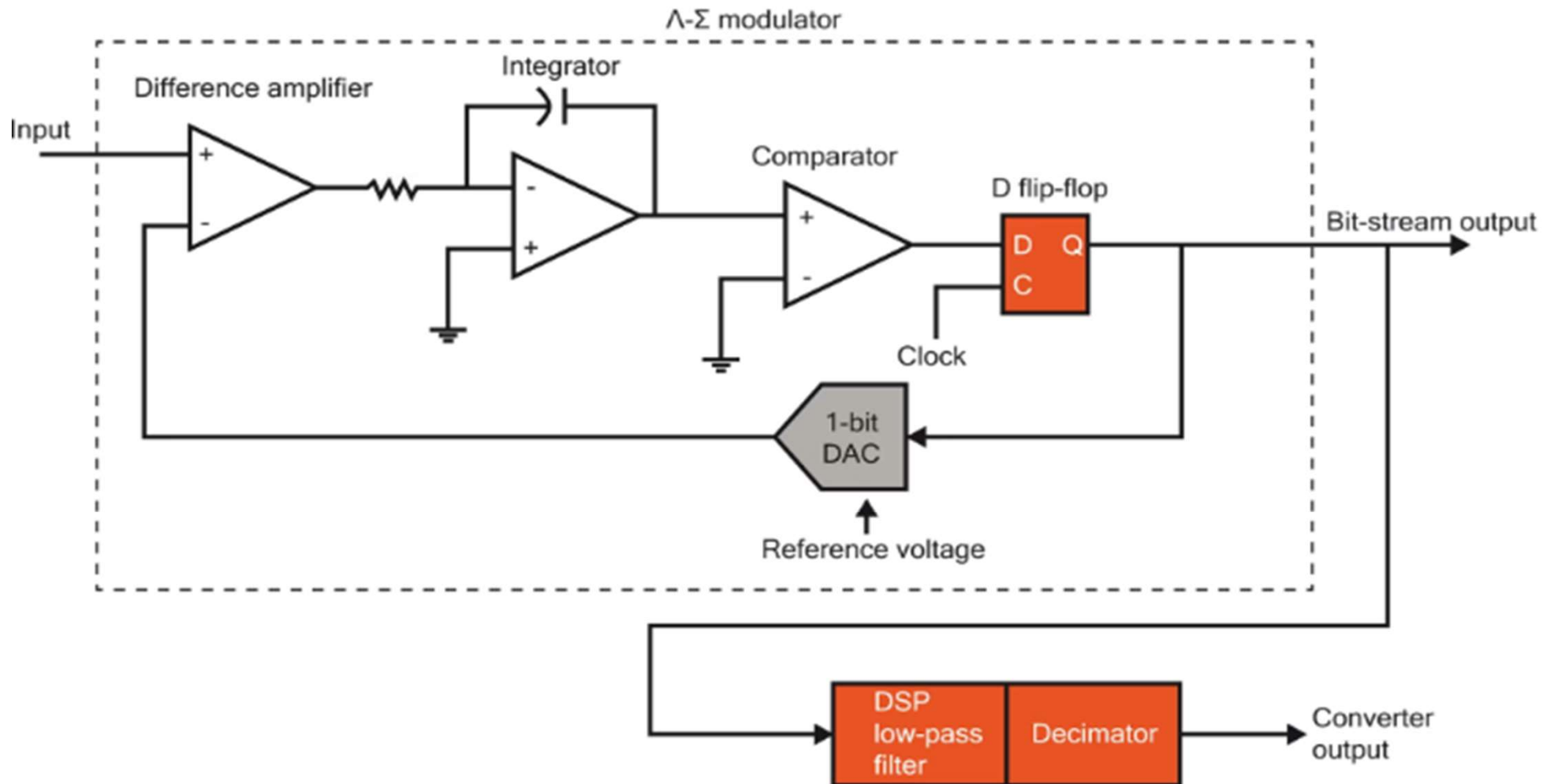
- Anti-aliasing filtering must be added externally
- Bit resolution and dynamic range limited compared to delta-sigma ADCs

Applications

Applications for SAR ADCs include DAQ systems, from low-end multiplexed ADC systems to higher speed single ADC per channel systems, industrial control and measurement, and CMOS imaging

Delta Sigma ADC:

ADC...



Delta Sigma ADC:

ADC...

Pros

- High-resolution output (24-bits)
- Over-sampling reduces quantization noise
- Inherent Anti-aliasing filtering

Cons

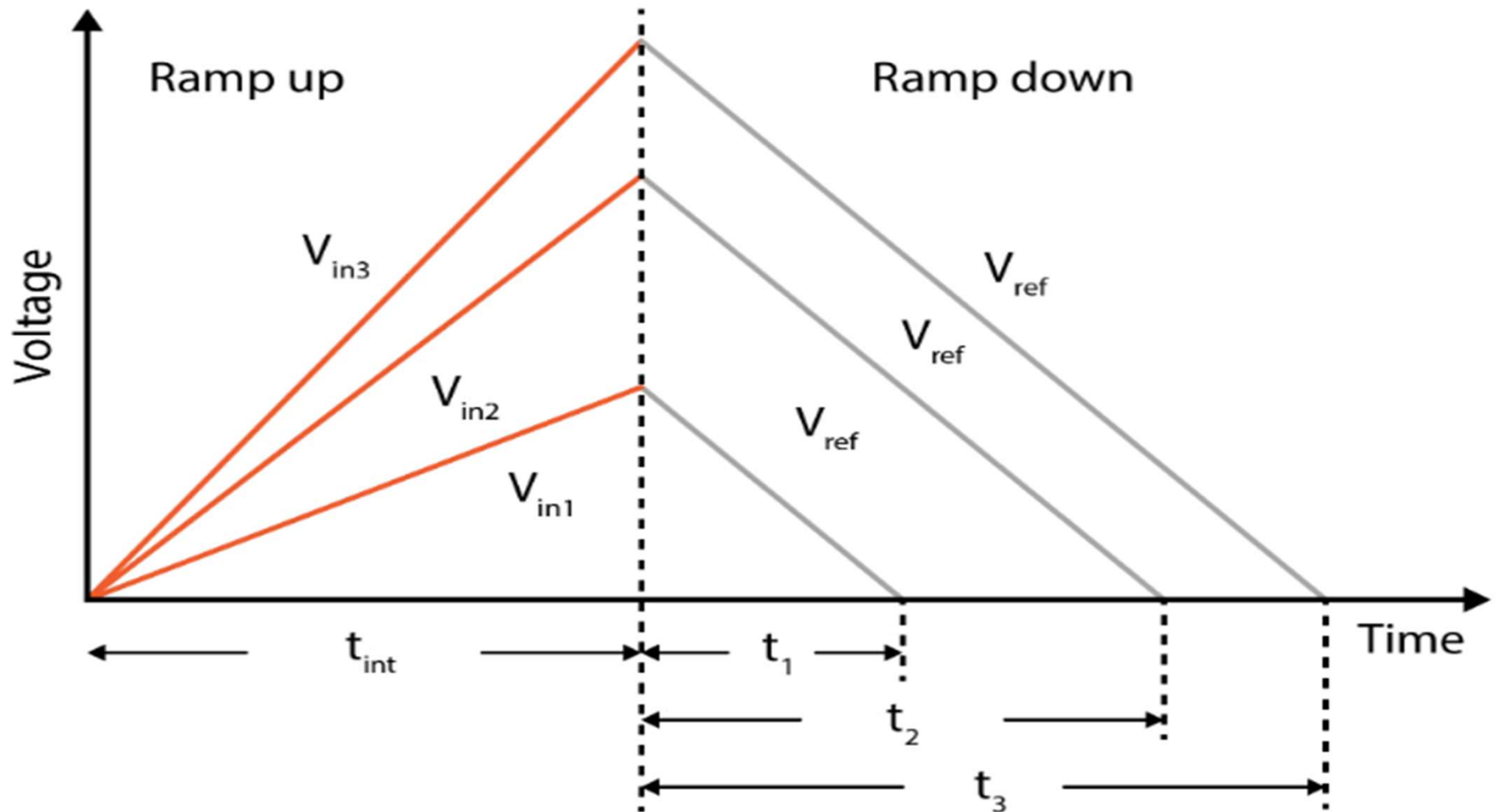
- Limited to around 200 kS/s sample rate
- Do not handle unnatural shape waveforms as well as SAR

Applications

Applications for Delta-sigma ADCs include data acquisition, especially noise and vibration, industrial balancing, torsional and rotational vibration, power quality monitoring, precision industrial measurements, audio and voiceband, and communications.

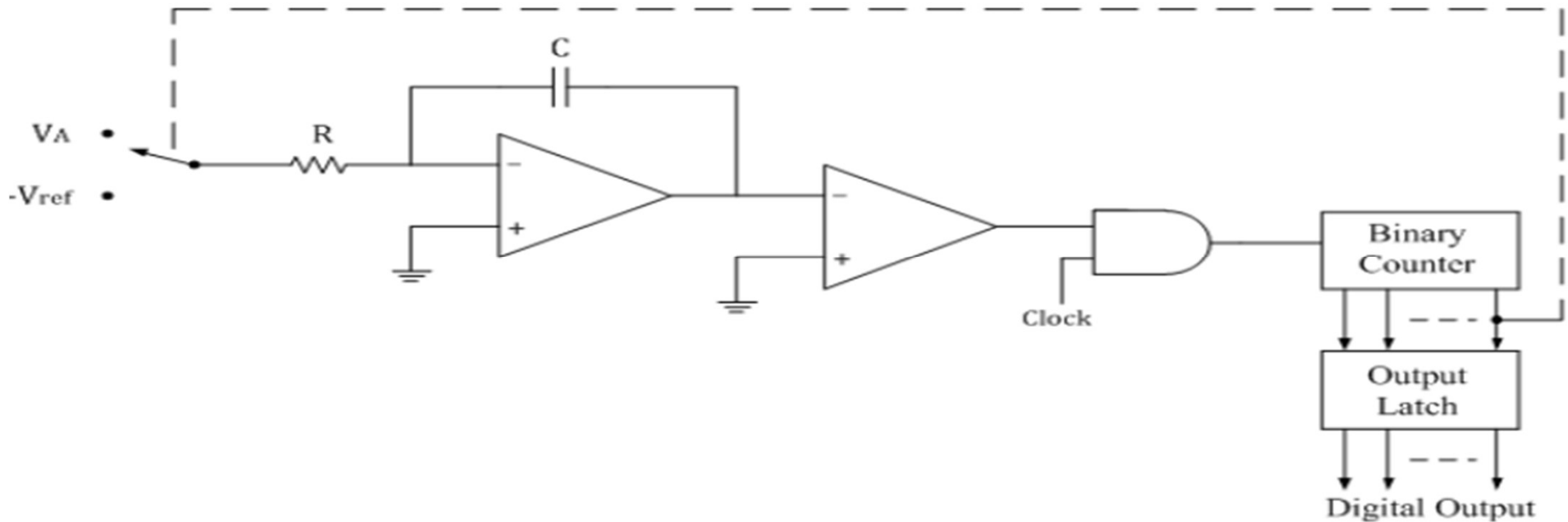
Dual Slope ADC:

ADC...



Dual Slope ADC:

ADC...



Pros

- Very precise and accurate measurements

Cons

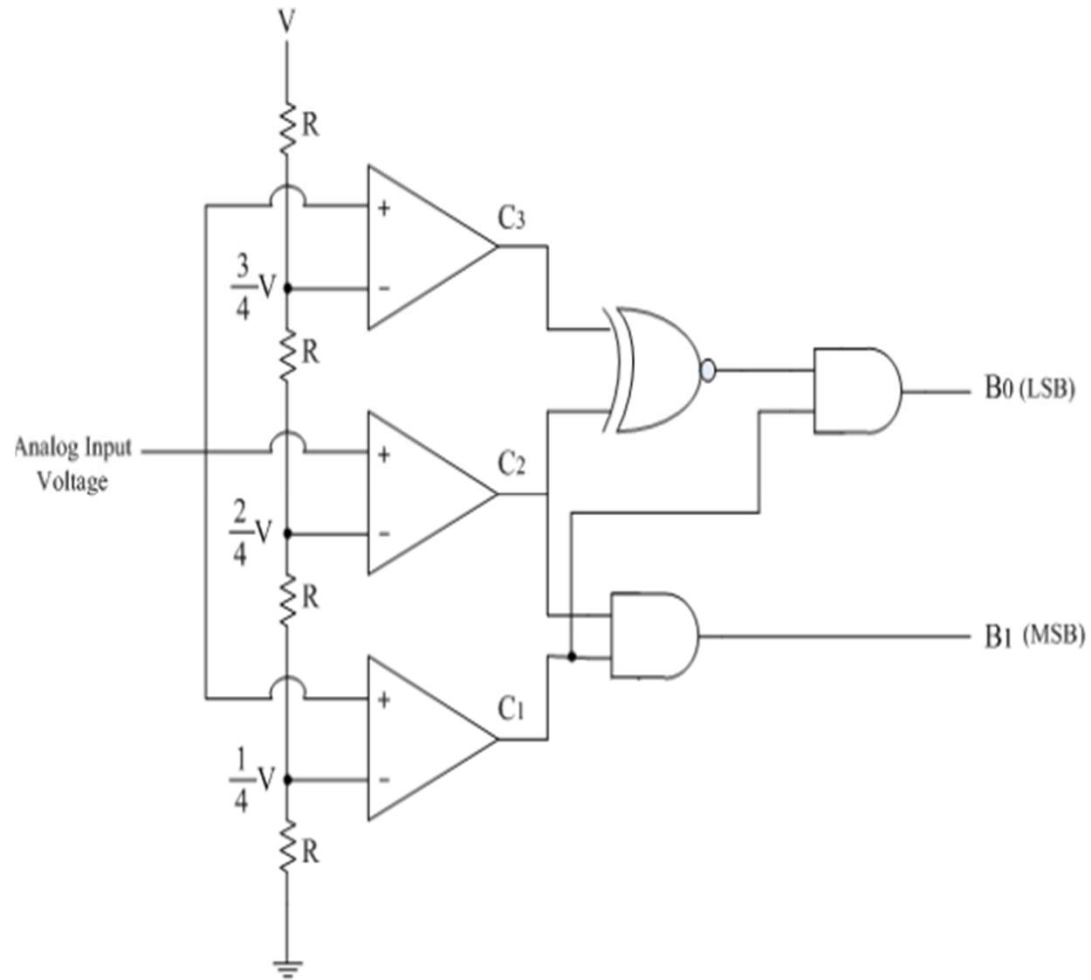
- Slow conversion time due to the ramp-up and ramp-down iteration

Applications

Applications for dual slope ADCs include handheld and benchtop multimeters.

Flash Type ADC:

ADC...



Flash Type ADC:

Analog Input Conditions	Comparator Outputs			Digital output	
	C ₁	C ₂	C ₃	B ₁	B ₀
$0 \leq V_{in} \leq \frac{V}{4}$	0	0	0	0	0
$\frac{V}{4} \leq V_{in} \leq \frac{2V}{4}$	1	0	0	0	1
$\frac{2V}{4} \leq V_{in} \leq \frac{3V}{4}$	1	1	0	1	0
$\frac{3V}{4} \leq V_{in} \leq V$	1	1	1	1	1

Pros

- Fastest ADC type
- Instant conversion without latency

Cons

- Circuit gets bigger and more power-consuming with each bit
- The resolution effectively limited to 8-bit

Applications

Applications for Flash ADCs include the fastest digital oscilloscopes, microwave measurements, fiber optics, RADAR detection, and wideband radio.

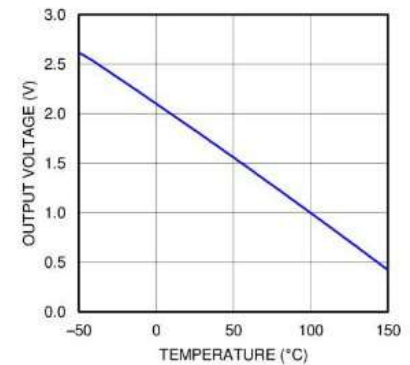
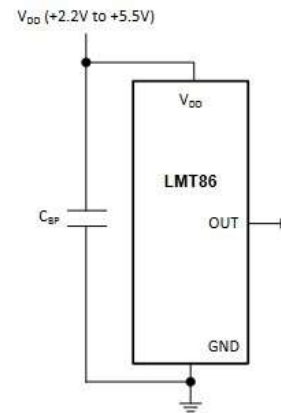
ADC...

ADC TYPE	PROS	CONS	MAX RESOLUTION	MAX SAMPLE RATE	MAIN APPLICATIONS
Successive Approximation (SAR)	Good speed/resolution ratio	No inherent anti-aliasing protection	18 bits	10 MHz	Data Acquisition
Delta-sigma ($\Delta\Sigma$)	High dynamic performance, inherent anti-aliasing protection	Hysteresis on unnatural signals	32 bits	1 MHz	Data Acquisition, Noise & Vibration, Audio
Dual Slope	Accurate, inexpensive	Low speed	20 bits	100 Hz	Voltmeters
Pipelined	Very fast	Limited resolution	16 bits	1 GHz	Oscilloscopes
Flash	Fastest	Low bit resolution	12 bits	10 GHz	Oscilloscopes

Questions

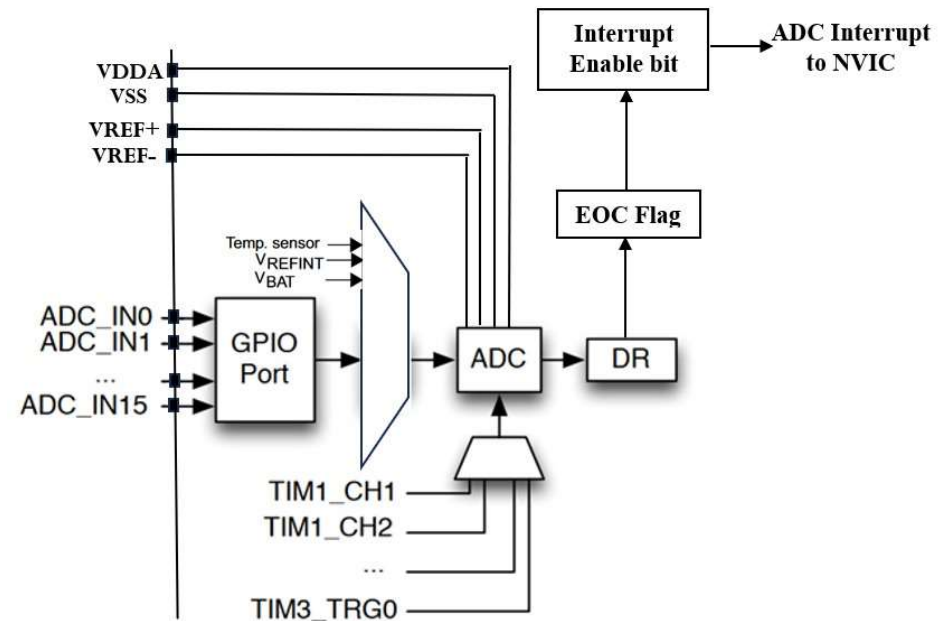
- STM32F407VG MCU is hosting two 12 bits ADCs with analog input range 0 V to 3.3 V. Calculate the resolution(step size) and quantization error.
- If ADC output is 0x100 with ADC specs as shown previous question, calculate the analog output voltage.
- Identify different ways to increase resolution of ADC.
- The figure below shows LMT86 Temperature sensor, corresponding IO characteristics and minimal specs. Comment on ADC selection.

Very Accurate: $\pm 0.4^{\circ}\text{C}$ Typical
Low 2.2-V Operation
Average Sensor Gain of $-10.9\text{ mV}/^{\circ}\text{C}$



ADC module in STM32F407VG MCU

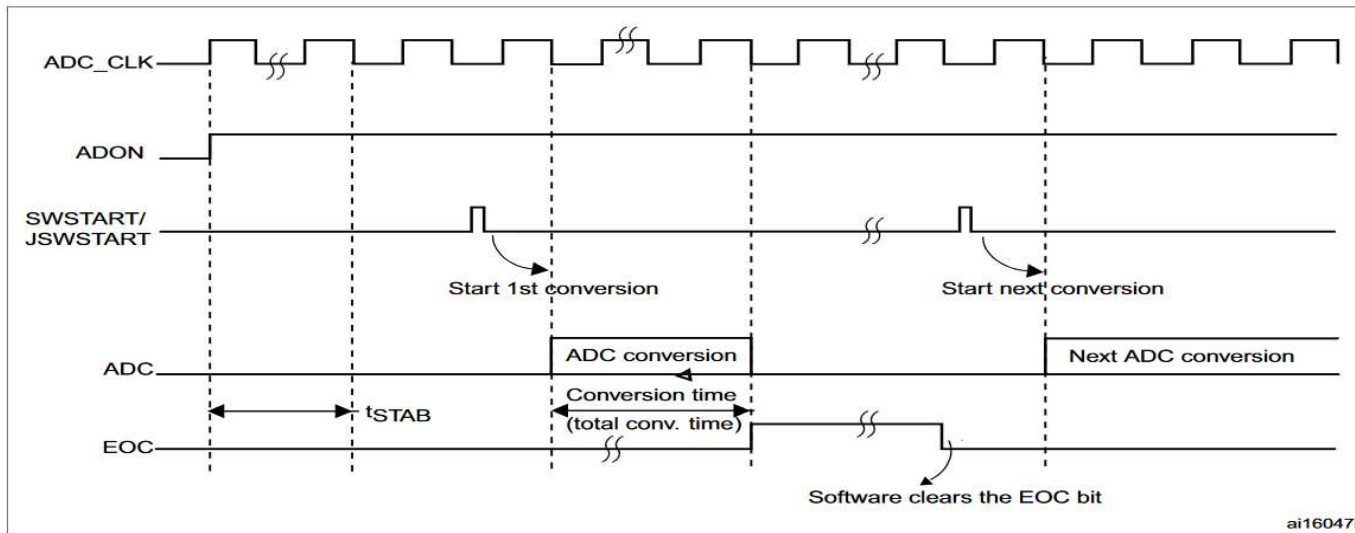
- The 12-bit ADC is a successive approximation ADC.
- It has up to 19 multiplexed channels allowing it to measure signals from 16 external sources and three internal sources(temp sensor, V_{REFINT} , V_{BAT})
- ADC module has a separate supply(V_{DDA}), which generally can be connected to the general MCU power supply.
- ADC can have an external reference source through V_{REF+} and V_{REF-} that can be brought through dedicated pins. In no particular case, the analog supply voltage may be as a reference internally.



- **Note:** V_{REF+} and V_{REF-} decide dynamic range of analog input signal($V_{REF-} \leq V_{IN} \leq V_{REF+}$)
- ***DR:** Data register
- ***EOC:** End of Conversion

ADC module in STM32F407VG MCU...

- Data Register(DR) is 12 bit hold the digital value. EOC(End of Conversion) flag is set indicate conversion complete.
- The analog to digital conversion can be triggered by software(setting SWSTART bit in ADC_CR2 register) or by Timer capture events after setting AD_ON bit in ADC_CR2 register.
- The timing associated with ADC module of STM32F407VG is shown below:



- t_{STAB} time for ADC stabilization

ADC module in STM32F407VG MCU...

Conversion time of ADC:

- The ADC clock(ADCCLK) is generated from the APB2 peripheral clock(PCLK2) divided by a programmable prescaler that allows the ADC to work at $PCLK2 / 2$, $/4$, $/6$ or $/8$ (default is 2).
- The total conversion time(for 12 bit resolution) is calculated as follows:

$$T_{conv} = \text{Sampling time} + 12 \text{ cycles}$$

Example:

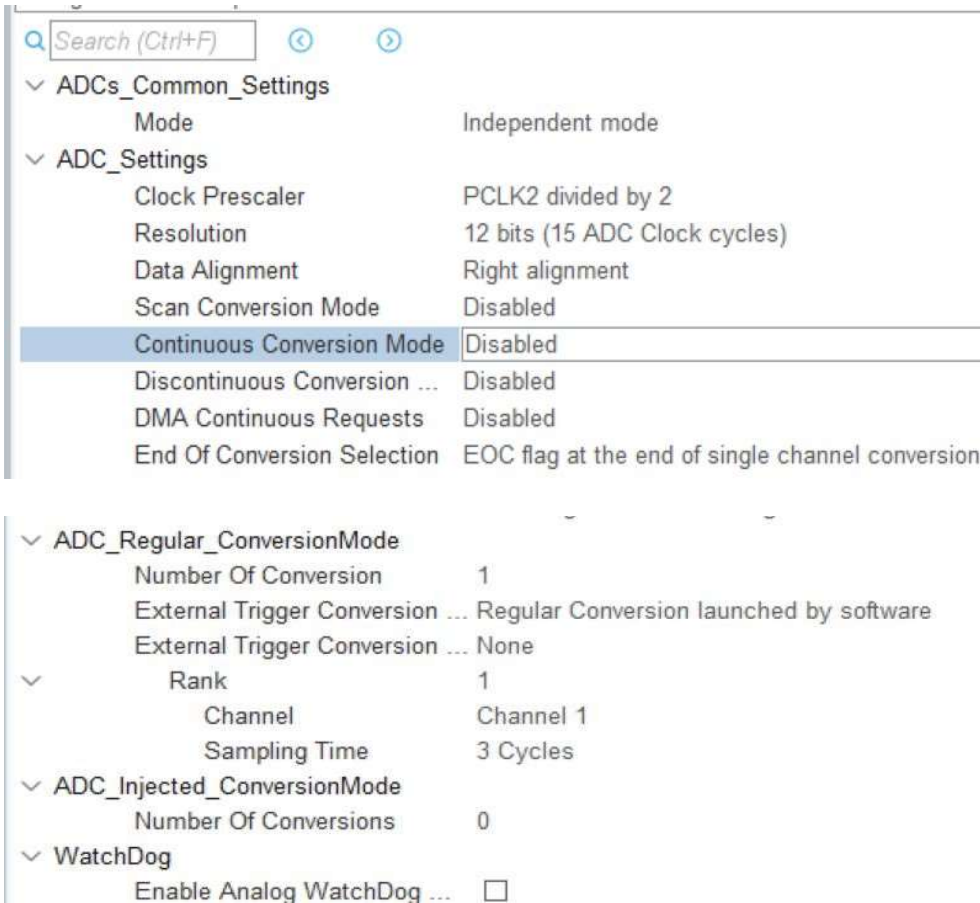
With ADCCLK = 30 MHz and sampling time = 3 cycles:

$$T_{conv} = 3 + 12 = 15 \text{ cycles} = 0.5 \mu\text{s with PCLK2 at 60 MHz}$$

- **Sampling time options:** 3 cycles, 15 cycles, 28 cycles, 56 cycles, 84 cycles, 112 cycles, 144 cycles, 480 cycles.

STM32CubeMx Configurations for ADC 1:

ADC module in STM32F407VG MCU...



HAL generated:

```
ADC_HandleTypeDef hadc1;    //ADC handler
```

```
void MX_ADC1_Init();
```

```
HAL_StatusTypeDef
```

```
HAL_ADC_Start(ADC_HandleTypeDef* hadc);
```

```
HAL_StatusTypeDef
```

```
HAL_ADC_PollForConversion(ADC_HandleTypeDef* hadc,  
uint32_t Timeout);
```

```
uint16_t HAL_ADC_GetValue(ADC_HandleTypeDef* hadc)
```

```
typedef enum  
{  
    HAL_OK          = 0x00U,  
    HAL_ERROR       = 0x01U,  
    HAL_BUSY        = 0x02U,  
    HAL_TIMEOUT     = 0x03U  
} HAL_StatusTypeDef;
```

Questions

Use STM32CubeMX to configure ADC1 of STM32F407VG MCU to following parameters:

Channel: IN1(PA1), PCLK2:60 MHz, ADCCLK:30MHz, Sampling time:3 ADCCLKs

Resolution:12 bits, Triggering: S/W controlled, Analog input range(dynamic range):0 V to 3.3 V(DC)

Write application code in Keil IDE to use ADC to analog to digital conversion and display equivalent digital value in debug window.

Questions

Use STM32CubeMX to configure ADC1 of STM32F407VG MCU to following parameters:

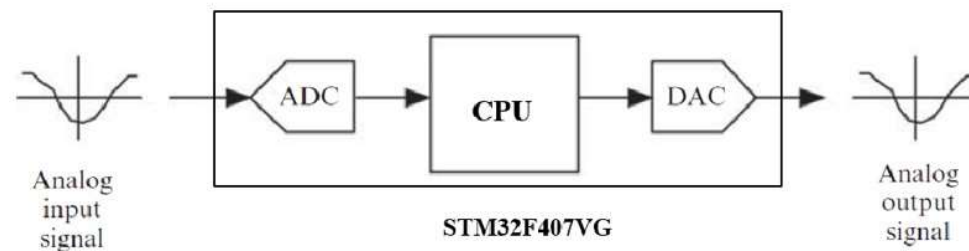
Channel: IN1(PA1), PCLK2:60 MHz, ADCCLK:30MHz, Sampling time:3 ADCCLKs

Resolution:12 bits, Triggering: S/W controlled, Analog input range(dynamic range):0 V to 3.3 V(DC)

Write application code in Keil IDE to use ADC to analog to digital conversion and generate PWM wave on GPIO pin PD9 in accordance to analog input voltage.

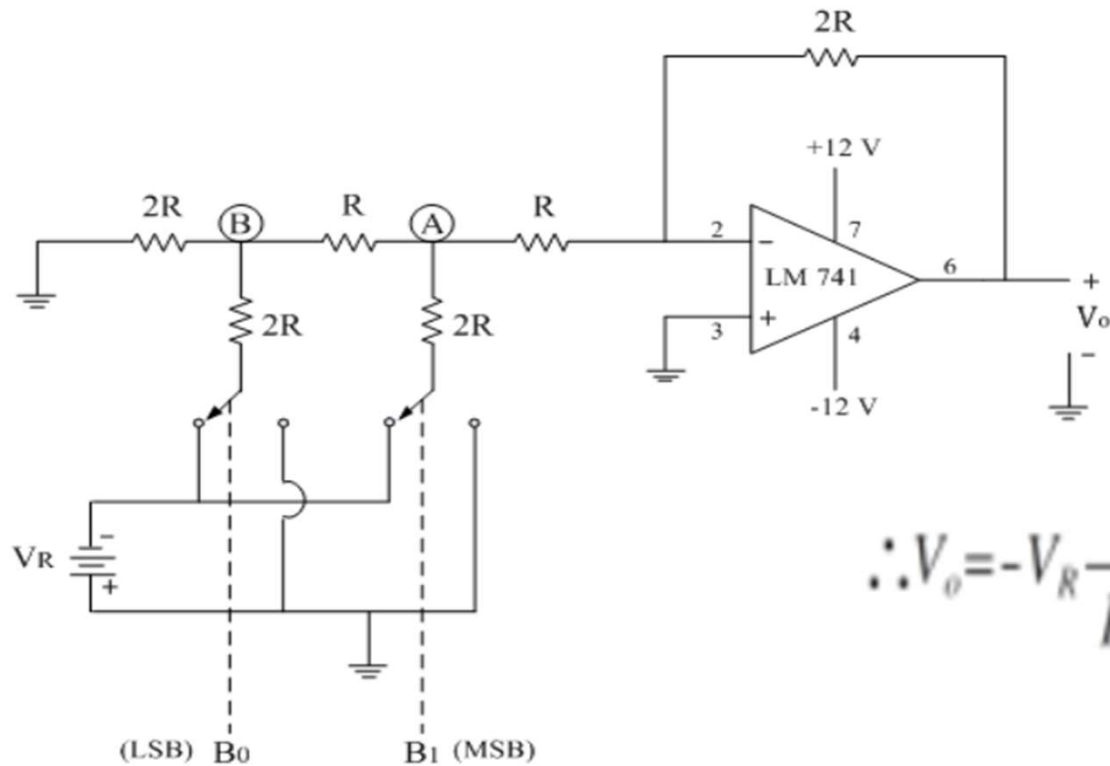
Digital to Analog Converter(DAC)

- DAC converts digital data into analog signal, and is used in variety of applications.
- DACs usually generates voltage equivalent of digital input data.
- STM32F407VG has two 12 bit DACs and can used with ADC to perform signal processing operations as shown in figure below.



- Digital inputs are converted to output voltages on a linear conversion between 0 and V_{REF+} .
- Two types of DAC: Weighted Resistor DAC and R-2R Ladder DAC.

R-2R LADDER DAC



$$V_o = -V_R \frac{R_f}{R} \left[\frac{B_1}{2^1} + \frac{B_2}{2^2} + \frac{B_3}{2^3} + \dots + \frac{B_n}{2^n} \right]$$

$$\therefore V_o = -V_R \frac{R_f}{R \times 2^n} [B_1 2^{n-1} + B_2 2^{n-2} + B_3 2^{n-3} + \dots + B_n 2^0]$$

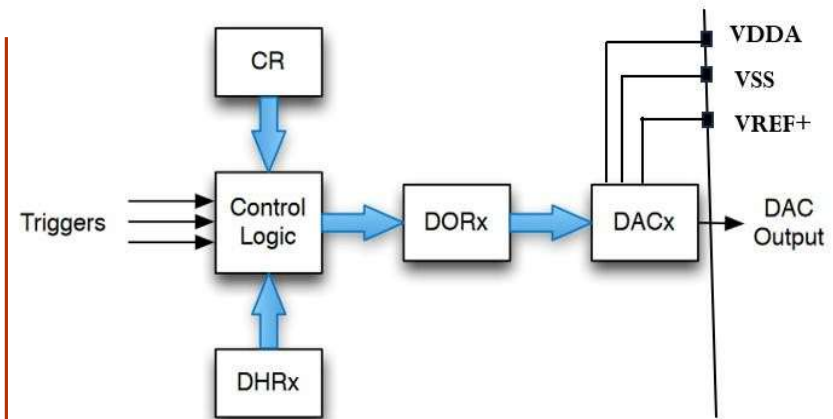
R-2R LADDER DAC

Sr. No.	Digital Input		Analog Output, $V_o(V)$
	B_1	B_0	
01	0	0	0
02	0	1	$\frac{V_R}{4}$
03	1	0	$\frac{2V_R}{4}$
04	1	1	$\frac{3V_R}{4}$

DAC module in STM32F407VG MCU

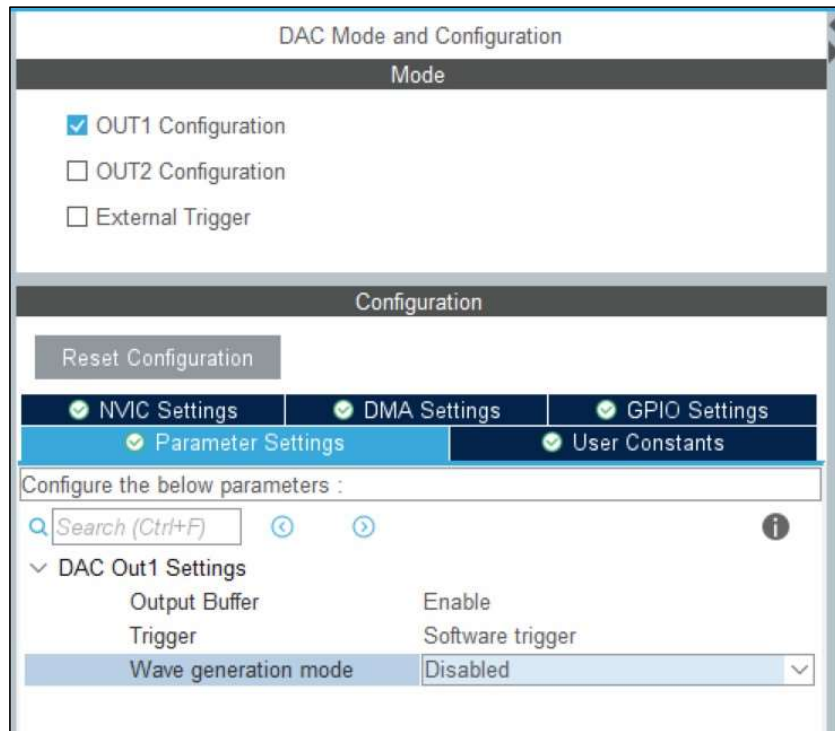
- Two DACs with separate output channels(OUT1-PA4 and OUT2-PA5).
- Each channel has separate control logic which is configured through a single control register (CR).
- Data to be converted by channel x are written to data holding register (DHRx). In response to a trigger event, DHRx is transferred to the data output register (DORx) and, after a *settling time*, the corresponding analog value appears at the output.
- Trigger events can include the trigger signals from various timers, an external interrupt, and software.
- The DAC output voltage(in volts) is linear between 0 and VREF + (3.3V on the discovery board) and is defined by the following equation:

$$DAC_{output} = V_{REF+} \times \frac{DOR}{4095}$$



- **Note:** Settling time is time it takes for the DAC's output voltage to reach and stay within a certain percentage of its final value after a digital input code is applied.

STM32CubeMx Configurations for DAC:



HAL generated:

DAC_HandleTypeDef hdac;

void MX_DAC_Init(**void**);

HAL_StatusTypeDef HAL_DAC_Start(**DAC_HandleTypeDef** *hdac, **uint32_t** Channel)

HAL_StatusTypeDef HAL_DAC_SetValue(**DAC_HandleTypeDef** *hdac, **uint32_t** Channel, **uint32_t** Alignment, **uint32_t** Data)

Channel: DAC_CHANNEL_1, DAC_CHANNEL_2

Alignment: DAC_ALIGN_12B_R, DAC_ALIGN_12B_L

Questions

Use STM32CubeMX to configure ADC1 and DAC of STM32F407VG MCU.

ADC Channel: IN1(PA1), PCLK2:60 MHz, ADCCLK:30MHz, Resolution:12 bits,
Triggering: S/W controlled

DAC channel: OUT 1(PA4), $t_{\text{SETTLING}}=3\text{ }\mu\text{s}$ (Digital to analog conversion time)

Set the conversion time of ADC to match the settling time of DAC.

Input: sine signal of 2 VPP with 1V DC offset and frequency: 1 Hz to 50 KHz in steps 3 KHz

Write application code in Keil IDE to use ADC for analog to digital conversion and DAC to reconstruct signal.